

Universidad
Nacional de Rosario



Sistemas operativos II

2026

Índice

1. Conceptos básicos **4**
1.1. ¿Que es un sistema operativo? . . . 4
1.2. Funciones . . . 4
1.3. Evolución de los sistemas operativos . . . 4
1.3.1. Proceso por lotes . . . 4
1.3.2. Sistemas en lotes con dispositivos de carga . . . 4
1.3.3. Sistemas multiprogramados . . . 5
1.3.4. Sistemas de tiempo compartido . . . 5
1.4. Organización de los sistemas operativos . . . 5

2. Relación con el hardware **6**
2.1. Jerarquía de almacenamiento . . . 6
2.2. Interrupciones y excepciones . . . 6
2.3. Acceso a memoria . . . 6
2.4. Llamadas al sistema . . . 6
2.5. Multiprocesamiento . . . 7

3. Procesos **7**
3.1. Conceptos básicos . . . 7
3.2. Estados de un proceso . . . 7
3.3. Información de un proceso . . . 7
3.4. Hilos . . . 7
3.5. Señales . . . 8
3.5.1. Problemas . . . 8
3.5.2. Señales y excepciones de hardware . . . 8
3.5.3. Funciones para señales . . . 8

4. Planificación de procesos **9**
4.1. Tipos de planificación . . . 9
4.2. Tipos de planificación . . . 10
4.3. Parámetros de respuesta . . . 10
4.4. Algoritmos de planificación . . . 11
4.4.1. FCFS(First come first serve) . . . 11
4.4.2. RR(Round robin) . . . 12
4.4.3. SRR(Selfish round robin) . . . 12
4.4.4. SPN(Shortest process next) . . . 12
4.4.5. PSPN(Preemptive shortest process next) . . . 12
4.4.6. HPRN(Highest penalty ratio next) . . . 12
4.4.7. FB(Multilevel feedback) . . . 13
4.4.8. Lotería . . . 13
4.4.9. PS(Priority scheduling) . . . 13
4.4.10. Métodos dependientes del sistema . . . 14
4.5. Inversión de prioridades . . . 14
4.6. Planificación en tiempo real . . . 14

5. Planificación en Linux **14**
5.1. Propósito . . . 14
5.2. Tipos y estados de tareas . . . 15
5.3. Clases de planificación . . . 15
5.4. Prioridades . . . 15

5.5. CFS(Completely Fair Scheduler) . . . 15
5.5.1. Compromisos . . . 16
5.5.2. CFS y prioridades . . . 16
5.5.3. Antes de CFS . . . 16

6. Administración de memoria **16**
6.1. Funciones y operaciones . . . 16
6.1.1. Espacio de direccionamiento . . . 16
6.1.2. MMU . . . 17
6.1.3. Caché . . . 17
6.1.4. Localidad . . . 17
6.1.5. Espacio en memoria de un proceso . . . 18
6.1.6. Resolución de direcciones . . . 18
6.2. Asignación de memoria contigua . . . 18
6.2.1. Fragmentación . . . 18
6.2.2. Compactacion . . . 19
6.2.3. SWAP . . . 19
6.3. Segmentacion . . . 19
6.3.1. Excepciones . . . 19
6.4. Paginacion . . . 20
6.4.1. Tamaño de los marcos . . . 20
6.4.2. La tabla de páginas y el TLB . . . 20
6.4.3. El tamaño de la tabla de páginas . . . 21
6.5. Memoria compartida . . . 21
6.5.1. Copy on write . . . 21
6.6. Memoria virtual . . . 21
6.6.1. Paginacion sobre demanda . . . 22
6.6.2. Reemplazo de páginas . . . 22
6.6.3. Anomalía de Belady . . . 23
6.6.4. FIFO(First in first out) . . . 23
6.6.5. OPT, MIN(Optimal, Minimal) . . . 23
6.6.6. LRU(Least recently used) . . . 23
6.6.7. MFU, LFU (Most frequently used, least frequently used) . . . 23
6.6.8. Aproximaciones a LRU . . . 24
6.7. Asignación de marcos . . . 24
6.7.1. Esquemas de asignación . . . 25
6.7.2. Ámbitos del algoritmo de reemplazo de páginas . . . 25
6.8. Hiperpaginación . . . 25

7. Memoria virtual en Linux **26**
7.1. Organización de la memoria . . . 26
7.2. Ubicación de memoria y gestión de memoria libre . . . 27
7.3. Reclamo de páginas . . . 27
7.4. OOM Killer . . . 28
7.5. `SWAP` en Linux . . . 28

8. Organización de archivos **28**
8.1. Conceptos básicos . . . 28
8.1.1. Concepto de archivo . . . 28
8.1.2. Tabla de archivos abiertos . . . 29
8.2. Acceso concurrente . . . 29
8.3. Tipos de archivo . . . 29

8.4. Estructura . . . 30
8.5. Métodos de acceso . . . 30
8.6. Archivos especiales . . . 30
8.7. Directorios . . . 30
8.7.1. Sistema de archivos plano . . . 31
8.7.2. Directorio de profundidad fija . . . 31
8.7.3. Directorios estructurados en árbol . . . 31
8.7.4. Directorios como grafos dirigidos . . . 31
8.7.5. Archivos especiales de directorio . . . 32
8.8. Control de acceso . . . 32
8.8.1. En Unix . . . 32
8.8.2. Listas de control de acceso . . . 32
8.9. Fallos y recuperación . . . 32
8.9.1. Datos y metadatos . . . 33
8.9.2. Verificación de integridad . . . 33
8.9.3. Actualizaciones suaves . . . 33
8.9.4. Bitacora o journaling . . . 33
8.9.5. Sistemas estructurados en bitácora . . . 34

9. Sistemas de archivos 34
9.1. Discos duros . . . 34
9.2. Estructura . . . 34
9.2.1. Componentes . . . 34
9.2.2. El directorio raíz . . . 35
9.3. Algunos sistemas de archivos . . . 35
9.3.1. Unix FS . . . 35
9.3.2. TAR(Unix) . . . 36
9.3.3. FAT . . . 36
9.4. Mejoras . . . 38
9.4.1. Haciendo crecer el sector lógico . . . 38
9.4.2. Algoritmo del ascensor . . . 38
9.4.3. Distribucion del superblock . . . 39
9.4.4. Eliminando secuencias dependientes . . . 39
9.4.5. Cambiando la estructura del directorio . . . 39
9.5. Algunos sistemas de archivos actuales . . . 39
9.5.1. ext4 . . . 39
9.5.2. B-tree FS . . . 39

1. Conceptos básicos

1.1. ¿Que es un sistema operativo?

Cuando hablamos de sistemas operativos en esta cátedra no hacemos referencia al sistema como lo ve o usa el usuario final. Nos interesan los detalles específicos de implementación. Estudiar esto es muy útil pues entender el funcionamiento básico de los sistemas operativos permite diseñar algoritmos y procesos que se ajustan al sistema donde van a ser ejecutados. Más específicamente podemos decir que un sistema operativo es un programa que ejecuta a otros programas, regulando su acceso a los distintos recursos y proveyendo interfaces para interactuar con los mismos.

1.2. Funciones

El sistema operativo es el único programa que interactúa directamente con el hardware de la computadora, sus principales funciones son:

- **Abstracción:** Los programas y procesos no deben tener que preocuparse de los detalles específicos del sistema. No deben tener que preocuparse de cómo acceder al hardware o de sus limitaciones. El sistema operativo se encarga de proporcionar una serie de abstracciones para que los programadores y usuarios puedan enfocarse en sus tareas.
- **Administración de recursos:** Un sistema operativo moderno tendrá a su disposición una gran cantidad de recursos(memoria, almacenamiento, poder de procesamiento, etc); Junto con los distintos procesos del sistema, que competirán por ellos. El sistema debe gestionar los recursos de forma que estos se asignen de forma efectiva y justa acorde a los objetivos del sistema.
- **Aislamiento:** En un sistema multiusuario y multitarea cada proceso y cada usuario no debe percibir que existen otros utilizando el mismo sistema. Idealmente su experiencia es la misma que si estuviera completamente solo. Para esto el sistema debe aislar los procesos y usuarios entre sí, para poder hacer esto es necesario que el sistema operativo utilice hardware específico para dicha protección.

1.3. Evolución de los sistemas operativos

1.3.1. Proceso por lotes

Los predecesores de los sistemas operativos modernos surgen en los años 50 al automatizar el procesamiento de diferentes programas. Los programadores entregaban su lote de tarjetas perforadas a los operadores del sistema que las alimentaban al sistema según correspondiese. En esta primera época el software monitor o de control se encargaba de asistir al operador en la carga de programas y las bibliotecas requeridas. Estos sistemas se fueron complejizando, implementando protecciones que evitaban la corrupción de otros trabajos, o que estos entrasen en un ciclo infinito. Estos monitores implican la modificación del hardware para realizar estas verificaciones, introducciones la característica básica de gestión de recursos.

1.3.2. Sistemas en lotes con dispositivos de carga

Una mejora natural a los sistemas antes mencionados es la invención del spool, un dispositivo de E/S que permite que una computadora de propósito específico, mucho menos costosa lea los datos y los almacene en un medio más rápido, la cinta magnética. Luego el el sistema principal utilizaba esta cinta magnética para E/S, eliminando un cuello de botella significativo.

1.3.3. Sistemas multiprogramados

A lo largo de la ejecución de un programa, inevitablemente, este va a encontrarse a la espera de algún recurso, generalmente procesamiento o E/S. Los sistemas multiprogramados buscaban maximizar el uso efectivo del procesador, ejecutando varios procesos al mismo tiempo. Este cambio de paradigma hizo necesario un cambio importante en el hardware, si bien se esperaba que cada programa fuese razonable con su uso de recursos resultó necesario implementar medidas de protección para garantizar esto.

1.3.4. Sistemas de tiempo compartido

Durante los años 60 la forma de interactuar con las computadoras cambió drásticamente. Los sistemas multiprogramados se convirtieron en sistemas interactivos y multiusuario. Esto llevó a un cambio fundamental sobre la forma en que se controlaban las multitareas:

- **Multitarea cooperativa:** Es la implementación de los sistemas multiprogramados, cada proceso tenía control total del CPU hasta que este hiciese una llamada al sistema o lo entregase voluntariamente. Como consecuencia un programa con bugs o un actor malicioso podía congelar todo el sistema.
- **Multitarea apropiativa:** En los sistemas de tiempo compartido el reloj del sistema interrumpe periódicamente los distintos procesos, entregando forzosamente el control al sistema operativo. Esto evita que un proceso acapare el CPU o congele el sistema.

Además fueron surgiendo de forma gradual las abstracciones que son comunes hoy en día, conceptos como archivos, directorios y librerías del sistema. Al mismo tiempo la velocidad del cambio entre tareas aumentó significativamente, pues en un sistema de tiempo compartido se busca dar la ilusión de uso exclusivo de la computadora.

1.4. Organización de los sistemas operativos

Si bien no hay líneas claras y reglas precisas para clasificar sistemas operativos podemos hacer ciertas distinciones de acuerdo a su estructura:

- **Monolíticos:** Históricamente la mayoría de los sistemas operativos han sido de este tipo. En ellos hay un solo proceso privilegiado(Justamente el sistema operativo) que opera en modo supervisor, donde están todas las rutinas para las tareas que realiza el sistema.
- **Microkernel:** En estos sistemas operativos el núcleo se mantiene lo más pequeño posible, delegando funcionalidades a procesos especiales sin privilegios. La principal ventaja de un sistema de este tipo es la simplificación de una gran cantidad de mecanismos de comunicación, lo que lleva a una mayor velocidad de ejecución. Al mismo tiempo estos sistemas siguen esquemas lógicos más simples, permitiendo implementaciones más elegantes que puede autorepararse con mayor facilidad.
- **Híbridos:** Como se mencionó anteriormente en este espacio no existen delimitaciones claras. Es muy común que un sistema emplee un esquema híbrido, por ejemplo sistemas que son mayormente monolíticos pero delegan ciertas responsabilidades a proceso nivel usuario como un sistema microkernel.

De esta distinción se desprende el hecho de que hay(al menos) dos modos de ejecución:

- **Modo núcleo o kernel:** Al ejecutarse en este modo un proceso tiene acceso a todos los recursos del sistema, se dice que un proceso en este modo es privilegiado. El sistema operativo se ejecuta en este modo.

- **Modo usuario:** Los procesos en este modo tienen restricciones, dependen del sistema operativo para realizar ciertas acciones, que pueden ser denegadas. Los programas nivel usuario se ejecutan de este modo.

2. Relación con el hardware

2.1. Jerarquía de almacenamiento

Las computadoras modernas siguen una arquitectura de Von Neumann, de forma resumida esta se basa en almacenar tanto los programas en ejecución como sus datos en la memoria primaria. Desgraciadamente este sistema tiene un problema importante, llamado el cuello de botella de Von Neumann, este se debe a que la velocidad de los procesadores ha sobrepasado con creces la de la memoria. Una respuesta parcial a este problema es la creación de una jerarquía de la memoria, donde en una misma computadora conviven diferentes memorias de velocidades y costos muy distintos. Comenzando con la memoria más rápida tenemos los registros, son parte del procesador, por lo que puede acceder a ellos mientras ejecuta instrucciones. Luego está la memoria caché del procesador, esta no es controlada directamente por el CPU, sino por los controladores de memoria, cargan los datos que el CPU va a necesitar o creen que va a necesitar. Luego tenemos la memoria principal, la RAM, aquí se almacenan los programas en ejecución y sus datos, es la más lenta de todas las memorias mencionadas, así como la más barata y abundante en un sistema común.

2.2. Interrupciones y excepciones

En el modelo de cómputo actual los procesos y el sistema deben ser capaces de responder a eventos que ocurren alrededor del sistema, son manejados mediante las interrupciones y excepciones. Cuando ocurre un evento que requiere la atención del sistema operativo el hardware encargado de procesarlo genera una interrupción. Esto detiene el proceso que se está ejecutando y la el control al sistema operativo, que ejecuta su rutina de manejo de interrupciones, para luego atender la interrupción. Las interrupciones tienen prioridades, de modo que una interrupción no interrumpa a otra de mayor importancia. El sistema puede elegir ignorar ciertas interrupciones, esto se llama enmascarar, hay ciertas interrupciones que no son enmascarables.

La diferencia entre interrupciones y excepciones es su origen, una interrupción es generada por causas externas al sistema, mientras que una excepción es un evento generado por un proceso, es decir que son síncronas. Vale recordar que las excepciones son interrupciones.

2.3. Acceso a memoria

Originalmente cuando un dispositivo de E/S necesitaba transferir datos a memoria realizaba una interrupción, esta era recibida por el procesador que debía detener su tarea actual para transferir el dato. Este proceso era ineficiente y creaba un cuello de botella. Para solucionar esto las computadoras modernas utilizan controladores DMA, son configurados por el CPU. Una vez configurados se encargan de la transferencia de datos, sin involucrar al procesador; Solo generan interrupciones cuando la transferencia termina u ocurre un error.

2.4. Llamadas al sistema

Como se mencionó anteriormente el sistema aísla los procesos entre sí y los abstrae de los detalles del sistema, esto implica que los procesos no privilegiados no tiene acceso directo a los dispositivos del sistema o ciertas acciones. Cuando un proceso debe realizar alguna acción privilegiada lo hace realizando una llamada al sistema, que es un tipo de excepción, es decir una interrupción síncrona. A grandes rasgos pueden clasificarse en: Control de procesos, manipulación de archivos, manipulación de dispositivos, acceso a información del sistema, comunicación.

2.5. Multiprocesamiento

El multiprocesamiento se da cuando un sistema posee más de un CPU. En este tipo de entornos el conjunto de procesadores se vuelve un recurso más que debe ser gestionado por el sistema operativo. Si bien el término multiprogramación tiende a usarse de forma intercambiable con multiprocesamiento son conceptos fundamentalmente distintos. Un sistema multiprogramado da la ilusión de que se están ejecutando múltiples procesos de forma concurrente, cuando en realidad alterna entre los distintos procesos en ejecución. Un sistema multiprocesado por el otro lado puede realmente ejecutar múltiples procesos de forma concurrente. Vale la pena notar que la mayoría de computadoras modernas tienen un esquema híbrido, donde hay múltiples procesadores, cada uno realizando multiprogramación.

3. Procesos

3.1. Conceptos básicos

En un sistema multiprogramado o de tiempo compartido un proceso es la imagen en memoria de un programa, junto con la información relacionada a su ejecución. Un programa es una entidad pasiva, un conjunto de datos e instrucciones, un proceso es una entidad activa, que proviene de un programa y puede actuar sobre el sistema.

3.2. Estados de un proceso

A lo largo de su vida, un proceso atraviesa una serie de estados de ejecución. Estos son:

- **Nuevo:** La creación de un proceso fue solicitada al sistema operativo, que está creando las estructuras y asignando los recursos necesarios.
- **Listo:** El proceso está listo para comenzar o continuar su ejecución, pero el sistema no le ha asignado ningún procesador.
- **En ejecución:** El proceso está siendo ejecutado por algún procesador.
- **Bloqueado:** El proceso está a la espera de algún evento para poder continuar su ejecución, aun si hubiese un procesador disponible no podría continuar.
- **Zombie:** El proceso terminó su ejecución, pero el sistema operativo debe realizar ciertas operaciones de limpieza antes de que este pueda terminar.
- **Terminado:** El proceso terminó de ejecutarse, sus estructuras están a la espera de ser limpiadas por el sistema operativo.

3.3. Información de un proceso

Toda la información relativa a un proceso es almacenada por el sistema operativo en una estructura llamada *process control block*(PCB). Contiene información sobre el estado del proceso, los registros, scheduling, memoria, E/S, etc.

3.4. Hilos

Es posible que un mismo proceso posea varios marcos de ejecución distintos, estos se conocen como hilos, threads o procesos ligeros. Todos los hilos de un proceso comparten el mismo espacio de memoria, así como los archivos y dispositivos abiertos. Estos se ejecutan de manera individual, cada uno tiene sus propios registros y pila. Los hilos más comunes son los de modo usuario, son controlados por un proceso y el sistema operativo no sabe de su existencia. Su ventaja principal es el rendimiento, al no tener que hacer un cambio de contexto para cambiar el hilo que está en ejecución. Sin embargo, esto

también implica que cualquier llamada al sistema por parte de uno de los hilos interrumpe la ejecución de todos los hilos, pues el sistema operativo no tiene conocimiento de los hilos. Por el otro lado existen los hilos nivel kernel, estos son transparentes al sistema y utilizan las funcionalidades que este provee. Esto soluciona el problema con las llamadas al sistema, pero implica una menor eficiencia, pues se debe involucrar al sistema operativo para hacer cambios entre threads.

3.5. Señales

Las señales son una forma de comunicación entre proceso usada en Unix y otros sistemas POSIX. Son notificaciones asíncronas enviadas a un proceso o hilo para notificarlo sobre un evento. Cuando se envía una señal el sistema operativo interrumpe la ejecución normal del destinatario para enviar la señal, esta interrupción puede ocurrir durante cualquier operación no atómica. Si el proceso registró previamente un manejador de señales entonces se ejecuta la rutina correspondiente, de lo contrario se ejecuta el manejador por defecto. Los primeros sistemas tenían un mecanismo de señales muy básico, luego UNIX incorporó un sistema más sofisticado que proveía más información sobre los eventos y permitía una mayor control sobre el mecanismo.

Los comportamientos definidos por defecto al llegar una señal son:

- Terminar el programa (`SIGKILL, SIGINT, SIGALRM`)
- Terminación anormal (`SIGABRT, SIGSEGV, SIGFPE`)
- Ignorar (`SIGCHLD`)
- Detener el programa (`SIGTSTP, SIGSTOP`)
- Continuar, si estaba detenido (`SIGCONT`)

Las señales que terminan un proceso cambian su estado a zombie, las que lo detienen a bloqueado y las que lo continúan a listo.

3.5.1. Problemas

En UNIX el manejo de señales es susceptible a problemas de concurrencia, como son asíncronas puede llegar una señal mientras se está ejecutando el handler de otra señal; También es posible que una señal interrumpa una llamada al sistema, generando un error. Para prevenir problemas adicionales los handlers de señales deben evitar los efectos secundarios: modificar `errno` o atributos globales, usar funciones no-reentrantes, etc.

3.5.2. Señales y excepciones de hardware

La ejecución de un proceso puede resultar en una excepción de hardware, por ejemplo si se intenta dividir por cero o acceder a una dirección de memoria inválida. Si el manejador de excepciones del núcleo no puede resolver el evento entonces envía una señal al proceso para que maneje la excepción.

3.5.3. Funciones para señales

Estas son las funciones definidas segun el estandard POSIX:

- `kill`: Envía la señal especificada al proceso dado, retorna 0 o -1.
  `int kill(pid_t pid, int, sig)`
- `getpid`: Retorna el `PID` del proceso llamante.
  `pid_t getpid(void)`
- `raise`: Autoenvía una señal al proceso que la llama.
  `int raise(int, sig)`

- `signal`: Define el manejador para la señal especificada. También se puede indicar que la señal debe tratarse con el manejador con `SIG_DFL` o que debe ser ignorada con `SIG_IGN`. Las señales `SIGKILL` y `SIGSTOP` no pueden ser interceptadas. Esta función está obsoleta, su implementación varía dependiendo de la versión de UNIX, usar `sigaction`.
  `sighandlert signal(int signum, sighandlert handler)`
- `sigaction`: Versión actualizada de signal, provee más información sobre la señal.
  `int sigaction(int signum, struct sigaction *act, struct sigaction *oldact)`
- `sigprocmask`: Modifica el conjunto de señales bloqueadas para el proceso, se usa para controlar problemas de concurrencia.
  `int sigprocmask (int how, const sigset *set, sigset *oset)`
- `alarm`: Programa un temporizador, cuando llega a cero envía la señal `SIGALRM` al proceso que llamó la función.
  `unsigned int alarm(unsigned int seconds)`

Dado que se ha tomado en algunos parciales se presentan las banderas de `sigaction` y su comportamiento de forma *muy* resumida, todos estos comportamientos son complejos y tienen asteriscos:

- `SA_NOCLDSTOP`: No notificar si el proceso hijo se detiene(stop) o continua(por `SIGCONT`).
- `SA_NOCLDWAIT`: No dejar zombies a los hijos, solo para `SIGCHILD`.
- `SA_NODEFER`: Permite que el handler sea reentrante.
- `SA_ONSTACK`: Utiliza un stack alternativo para el manejador de la señal.
- `SA_RESETHAND`: Vuelve al handler default luego de invocar al handler.
- `SA_RESTART`: Relanza las syscalls interrumpidas por la señal.
- `SA_SIGINFO`: Indica que se debe utilizar un handler *simple*, similar al utilizado por `signal`.

4. Planificación de procesos

4.1. Tipos de planificación

La planificación de proceso se refiere a cómo determina el sistema operativo el orden en que le dará acceso al procesador a los distintos procesos que lo solicitan, así como también las políticas para asegurar que el uso del mismo no sea excesivo por parte de ningún proceso. Hay tres tipos principales de planificación:

- **A largo plazo:** Decide qué procesos serán los siguientes en ser iniciados, este tipo de planificación era más frecuente en los sistemas por lotes. En los sistemas interactivos, casi la totalidad de los que se usan hoy en día, este tipo de planificación no se efectúa, dado que normalmente el usuario indica qué procesos iniciar. Hace los pasos entre: `nuevo -> listo`.
- **A mediano plazo:** Decide qué procesos es conveniente bloquear en determinado momento, ya sea por escasez/saturación de algún recurso o porque están realizando una solicitud que no puede satisfacerse. También se encarga de tomar decisiones sobre los procesos a medida que entran y salen del estado bloqueado. Hace los pasos entre `ejecución -> bloqueado` y `bloqueado -> listo`.
- **A corto plazo:** Decide cómo se debe distribuir el uso del CPU momento a momento, se realiza decenas de veces por segundo. Es el encargado de elegir cual de los procesos listos para ejecutar tendrá uso del CPU. Hace los pasos entre `listo -> en ejecución`.

4.2. Tipos de planificación

Los procesos normalmente alternan entre rafagas, en las que realizan principalmente cómputo interno y otras en las que la atención está en hacer E/S. Los procesos largos son los que por mucho tiempo han estado listos o en ejecución, es decir que están en una rafaga limitada por el CPU. Por el otro lado los cortos son los que en este momento están en una rafaga limitada por E/S y requieren atención ocasional del procesador, o tienden a estar bloqueados esperando eventos. Por lo general se le da un tratamiento preferente a los procesos cortos, especialmente a los interactivos.

4.3. Parámetros de respuesta

Mientras que los conceptos de planificación permanecen iguales los tiempos de respuesta cambian constantemente, a medida que evoluciona el hardware. Por lo tanto se emplean unidades abstractas, *ticks* y *quantums* para medir los tiempos de respuesta. El primero representa una fracción de tiempo durante la cual se puede realizar trabajo útil, es decir usar el CPU sin interrupción(Ignorando las interrupciones de E/S). La duración de un tick está definida por una interrupción periódica generada por el temporizador del sistema. En Linux dura un milisegundo, en Windows entre 10 y 15. Los quantums representan el tiempo mínimo que se permite a un proceso el uso del CPU, se define como una cierta cantidad de ticks. En linux es entre 10 y 200, en Windows entre 2 y 12 ticks.

Definimos entonces para un proceso p y un tiempo de ejecución t, las métricas usadas para medir el comportamiento del sistema bajo un planificador dado:

- **Tiempo de respuesta(T):** Tiempo total que requiere el proceso para completar el trabajo pendiente. Se comienza a contar desde que el proceso está listo. Con un único procesador y proceso $T = t$.
- **Tiempo en espera($E = T - t$):** Cuánto tiempo p está listo y esperando para ejecutar, es tiempo perdido. Desde el punto de vista de p se desea que sea cero.
- **Proporción de penalización($P = \frac{T}{t}$):** Proporción en la que fue penalizado el proceso, cuánto del tiempo de respuesta pasó sin ejecutarse.
- **Proporción de respuesta($R = \frac{t}{T}$):** Inverso de P, cuánto tiempo del tiempo de respuesta p se puedo ejecutar.
- **Frecuencia de llegada(α):** Promedio de procesos nuevos en la cola de procesos listos en un rango definido.
- **Frecuencia de servicio requerido(β):** Cuantos procesos terminan en un rango definido.
- **Valor de saturación($\rho = \frac{\alpha}{\beta}$):** Cuando es cero nunca llegan procesos nuevos, si es uno los procesos son despachados al mismo ritmo al que llegan. Si es mayor a uno los procesos llegan más rápido de lo que la computadora puede atenderlos, la cola de procesos listos crece, se degrada la calidad de servicio.
- **Tiempo de kernel:** Tiempo que pasa el sistema en espacio kernel, no se cuenta al calcular el tiempo de CPU de un proceso.
- **Tiempo de sistema:** Tiempo que pasa un proceso en espacio kernel, cuando el sistema está atendiendo sus llamadas al sistema.
- **Tiempo de usuario:** Tiempo que pasa un proceso en modo usuario, es decir ejecutando su programa.
- **Tiempo de uso del procesador:** Tiempo durante el cual el CPU ejecuta instrucciones del proceso, sean en modo usuario o kernel.

- **Tiempo idle:** Tiempo en el que la cola de procesos está vacía y el CPU no está ejecutando instrucciones.
- **Utilización del CPU:** Porcentaje del tiempo en el que el CPU está realizando trabajo útil, en sistemas reales está entre 40 y 90 %.

4.4. Algoritmos de planificación

Analizamos ahora los objetivos de la planificación a corto plazo así como algunos algoritmos que intentan resolver el problema. A grandes rasgos los objetivos de este tipo de planificación(Notando que pueden ser contradictorios) son:

- Ser justo, debe tratar de la misma forma a todos los procesos con características similares y no postergar ninguno de ellos indefinidamente.
- Maximizar el rendimiento, dar servicio a la mayor parte de procesos por unidad de tiempo.
- Ser predecible, un mismo trabajo debe tomar aproximadamente la misma cantidad de tiempo sin importar la carga del sistema.
- Minimizar la sobrecarga, el tiempo que el algoritmo gasta en burocracia debe mantenerse al mínimo.
- Equilibrar el uso de recursos, favorecer a los procesos que utilizan recursos utilizados y penalizar a los que utilicen recursos sobreutilizados.
- Enviar la inanición, aumenta la prioridad de procesos más viejos, permitiendo que se ejecuten en tiempo y forma.
- Favorecer el uso esperado del sistema, en un sistema interactivo maximizar la prioridad de procesos que sirvan a las solicitudes iniciadas por los usuarios.
- Dar preferencia a los procesos que podrían bloquear el sistema, si un recurso de baja prioridad está utilizando un recurso del sistema por el que otros procesos están esperando, favorecer a que este termine para que libere el recurso.
- Favorecer a los procesos con comportamiento deseable, si un proceso causa muchas demoras se lo puede penalizar para evitar que se degrade el rendimiento global del sistema.
- Degradarse suavemente, si $\rho > 1$ se debe tratar de responder con la menor penalización a procesos preexistentes al momento de exceder el umbral.

4.4.1. FCFS(First come first serve)

El esquema más simple posible, es un mecanismo cooperativo, cada proceso se ejecuta en el orden en que llegó, hasta que suelta el control. El despachador es una cola *FIFO*. Notas:

- Minimiza la cantidad de cambios de contexto.
- Reduce la sobrecarga administrativa(Elegir al siguiente es O(1)).
- No requiere hardware de apoyo.
- Favorece a los procesos largos.
- No utiliza información sobre los procesos.
- Los últimos en llegar tienen un comportamiento inaceptable.
- Es inadecuado para trabajos interactivos.

4.4.2. RR(Round robin)

Intenta dar una buena respuesta para procesos cortos y largos, es un mecanismo apropiativo. Cada proceso en la lista de procesos listos solo puede ejecutarse durante una cierta cantidad de *quantums* (q), luego será interrumpido y puesto al final de la lista si es que no ha terminado. El valor de q puede ajustarse, con q arbitrariamente grande se vuelve **FCFS**, idealmente el *quantum* debe mantenerse inferior a la duración promedio de 80 % de los procesos.

4.4.3. SRR(Selfish round robin)

Versión egoísta de **RR**, busca favorecer a los procesos que ya se han ejecutado, a costa de los recién llegados. Utiliza dos colas, una de procesos nuevos y otra de procesos aceptados, los procesos ingresan a la cola de procesos nuevos, con prioridad 0. Avanzan a la cola de aceptados cuando su prioridad alcanza la de algún proceso aceptado. Los elementos aceptados son manejados por un esquema **RR**. El incremento de las prioridades es controlado por dos parámetros a y b, determinan en cuánto aumenta la prioridad de los procesos en la cola de procesos nuevos y aceptados, respectivamente. Notas:

- Si $\frac{b}{a} < 1$ todo proceso nuevo eventualmente será aceptado.
- Si $\frac{b}{a} > 1$ se convierte en **FCFS**.
- Si $\frac{b}{a} = 0$ se convierte en **RR**.
- Utiliza información intrínseca.
- Favorece a los procesos largos sin bloquear completamente a los cortos.
- Los parámetros se deben ajustar correctamente para un buen funcionamiento.

4.4.4. SPN(Shortest process next)

Cuando no se puede implementar multitarea apropiativa pero se requiere un algoritmo más justo se puede usar información sobre lo que un proceso va a demorar en el futuro. Cómo obtener esta información es muy difícil se busca caracterizar las necesidades del proceso analizando como se ha comportado. Se utiliza un promedio exponencial: $e_p^{'} = f.e_p + (1 - f)q$ donde q es los *quantums* que p utilizó en su última ejecución, se elije el proceso con menor e_p. Notas:

- Favorece los procesos cortos.
- Un proceso largo puede sufrir inanición, especialmente si ρ es cercano o mayor a 1.
- Utiliza información intrínseca.

4.4.5. PSPN(Preemptive shortest process next)

Utiliza el mismo mecanismo que **SPN**, con la diferencia de que es apropiativo. Si llega un nuevo proceso con e_p menor al del proceso en ejecución el proceso en ejecución será reemplazado. Obtiene mejores promedios que **SPN** de forma consistente pues mantiene la lista corta al despachar primero los procesos cortos

4.4.6. HPRN(Highest penalty ratio next)

Si no se dispone de multitarea apropiativa los mecanismos **FCFS** y **SPN** no son ideales, este algoritmo intenta ser más balanceado. Cada proceso tiene un índice de penalización P, con un valor inicial de 1. Cada vez que un proceso es obligado a esperar por un tiempo w a otro proceso, su valor de P se actualiza a $P = \frac{w+t}{t}$. El proceso con mayor P se elige como activo. Notas:

- Si $\rho < 1$ ningún proceso sufre de inanición.
- Es costoso calcular P para cada proceso, especialmente si la lista es larga.
- Utiliza información intrínseca.

4.4.7. FB(Multilevel feedback)

Utiliza varias colas, es muy usado hoy en día. Se tienen múltiples colas, el despachador elige el proceso al frente de la cola de mayor prioridad. Tras un número predeterminado de ejecuciones la prioridad de un proceso es degradada, se lo pasa a la cola de prioridad inmediatamente inferior, si es que existe. El tiempo de ejecución permitido para un proceso puede ajustarse de acuerdo a la cola en la que esté. Notas:

- Implementa multitarea apropiativa.
- Con los ajustes adecuados es mejor que **RR**.
- Se puede utilizar *aging* para evitar inanición, evitando que procesos pasen demasiado tiempo sin ejecutarse.
- Utiliza información intrínseca.
- Cada cola puede utilizar un esquema diferente para elegir el próximo proceso.
- Es relativamente complejo de implementar, especialmente si se permiten distintos esquemas.

4.4.8. Lotería

En este esquema cada proceso tiene un número de boletos, cada boleto representa una posibilidad de salir ganador. Para elegir al próximo proceso se elige un número al azar y se otorga el siguiente quantum al proceso ganador. La cantidad de boletos de un proceso es constante. Las prioridades se representan mediante la cantidad de boletos, a más boletos mayor prioridad. Considera que los procesos pueden cooperar entre sí, por ejemplo si un proceso A espera un recurso de B, puede donar sus boletos para aumentar las probabilidades de que B sea ejecutado. Notas:

- Implementa multitarea apropiativa.
- Evita inanición, pues todo proceso esperando ejecutarse puede ser elegido, y eventualmente lo será.
- Presenta variabilidad a la hora de asignar el CPU.
- No es determinista.

4.4.9. PS(Priority scheduling)

Este esquema puede o no ser apropiativo. En ambos casos se elige el proceso de mayor prioridad para ejecutar, estas prioridades pueden ser internas o externas, dinámicas o estáticas. El caso más sencillo es el no apropiativo, se espera a que el proceso en ejecución termine para elegir su reemplazo. Por el otro lado, en el caso apropiativo el proceso en ejecución será interrumpido si llega o se vuelve listo un proceso de mayor prioridad. Notas:

- Puede producir inanición de los procesos de baja prioridad, se puede mitigar este problema usando *aging*.
- Es sencillo de implementar.

4.4.10. Métodos dependientes del sistema

Si bien no existe un esquema de planificación ideal se puede tomar una decisión en base a las características del sistema:

- Si en promedio, los procesos son cortos y $\rho < 1$ se deberían utilizar métodos con poca sobrecarga administrativa, como **FCFS**, **SPN** o **RR** con q grande.
- Si la cola de tareas es muy larga se debe usar **RR** con q pequeño o **PSPN**, que garantizan una buena distribución del CPU.
- Siempre es una buena opción utilizar **RR**, donde q se actualiza periodicamente dependiendo de la cantidad de procesos. Si hay pocos procesos se minimiza la cantidad de cambios de contexto con q grande.

4.5. Inversión de prioridades

Este problema ocurre cuando un proceso que necesita acceder a un recurso debe esperar porque este está siendo usado por un proceso de menor prioridad. Este problema puede mitigarse de dos formas:

- **Techo de prioridad:** Se le asigna una prioridad máxima al recurso compartido, la más alta entre cualquier proceso que pueda acceder a él. Cuando un proceso adquiere el recurso su prioridad se eleva temporalmente a la máxima, cuando libera el recurso, su prioridad se restaura al valor original.
- **Herencia de prioridades:** El proceso de menor prioridad, hereda temporalmente la prioridad del proceso esperando al recurso. Cuando libera el recurso su prioridad se restaura al valor original.

4.6. Planificación en tiempo real

Hay procesos que requieren garantías de tiempo, necesitan un cierto tiempo de ejecución antes de un tiempo límite, se conocen como procesos de tiempo real. Los algoritmos vistos hasta el momento no son adecuados para este tipo de procesos. La opción ideal para manejar procesos de tiempo real es el uso de tiempo real duro, donde el sistema puede garantizar que un proceso tendrá el suficiente tiempo de ejecución en un intervalo dado; No es posible hacer esto en sistemas de propósito general con hardware estándar. Para solventar estas necesidades en sistemas de uso general se usa tiempo real suave, donde los procesos de tiempo real reciben un tratamiento prioritario sobre los procesos comunes, generalmente se implementa con un esquema similar a retroalimentación multinivel, con las siguientes peculiaridades:

- La cola de tiempo real recibe prioridad sobre todas las otras.
- La prioridad de un proceso de tiempo real no se degrada.
- Los demás procesos nunca llegan a la cola de tiempo real por un proceso automático.
- La latencia de despacho debe ser mínima.

5. Planificación en Linux

5.1. Propósito

El planificador de Linux tiene dos objetivos principales. El primero es poder adaptarse lo mejor posible a cualquier entorno de uso, sin requerir demasiado poder de cómputo, pero logrando buen rendimiento. El segundo es optimizar el rendimiento de los procesadores pero sin dejar de lado a las tareas interactivas, lo que implica que las tareas vinculadas a la E/S no pueden ser demoradas por mucho tiempo.

5.2. Tipos y estados de tareas

Basado en los objetivos anteriores Linux intenta hacer una distinción clara basada en el tipo de las tareas:

- **Interactivas:** Suelen estar en estado bloqueado la mayor parte del tiempo, esperando algún evento externo. Cuando se recibe el evento se debe reaccionar lo más rápido posible.
- **Batch:** Hacen un uso intensivo del CPU, pueden permitirse un poco de demora.

El ciclo de una tarea es similar al de cualquier sistema UNIX, con algunas diferencias:

- `TASK_RUNNING`: Esta lista *o* en ejecución, notar que se usa el mismo valor para ambas.
- `TASK_INTERRUPTABLE`: Está bloqueada, pero se le pueden retirar recursos.
- `TASK_UNINTERRUPTABLE`: Está bloqueada, no se le pueden retirar recursos, generalmente se utiliza para funciones críticas y *syscalls*.
- `TASK_STOPPED`: Bloqueada por una señal.
- `TASK_TRACED`: Bloqueada, esta siendo depurada en un *debugger*.
- `EXIT_ZOMBIE`: Zombie.
- `EXIT_DEAD`: Terminada.

El planificador descrito a continuación se encarga de las tareas en estado `TASK_RUNNING`.

5.3. Clases de planificación

Como se vio anteriormente las tareas de tiempo real requieren una atención y tiempo de respuesta mayor a los procesos regulares, para esto utilizan dos colas especiales:

- `SCHED_FIFO`: Las tareas en esta clase solo pueden ser reemplazadas por tareas de mayor prioridad, de lo contrario se ejecutan hasta terminar o bloquearse.
- `SCHED_RR`: Se utiliza un esquema **RR** con *quantum* fijo.

El resto de las tareas tienen la clase `SCHED_OTHER`, analizaremos principalmente este tipo de tareas.

5.4. Prioridades

Cada tarea tiene una prioridad estática que se puede ajustar con las funciones `nice` y `set_priority`. Los procesos hijos e hilos creados por una tarea heredan su prioridad. Internamente el planificador le asigna una prioridad dinámica a cada tarea. Está basada en la prioridad estática, pero varía en función del tipo de tarea y el estado del sistema.

Linux utiliza prioridades entre 0 y 139, donde 0 es la mayor, los valores entre 0 y 99 están reservados para tareas de tiempo real; Es decir tareas cuya clase es `SCHED_FIFO` o `SCHED_RR`.

5.5. CFS(Completely Fair Scheduler)

El scheduler usado por Linux hoy en dia, ya se está preparando su reemplazo **EEVDF**. Este algoritmo pretende simular el comportamiento de una CPU ideal, divide el poder de cómputo equitativamente entre todas las tareas listas, dado que en un CPU real no pueden ejecutarse dos tareas en simultáneo se opta por intentar dividir el tiempo de uso del CPU.

Para hacer esto se usa una técnica simple basada en el conteo, llamada *virtual runtime*, en la práctica indica cuanto tiempo de ejecución tuvo una tarea, normalizado sobre el número total de tareas. Cuando ocurre una decisión de planificación **CFS** selecciona el proceso con menor *virtual runtime* para ejecutarse a continuación. Las tareas en estado *runnable* se almacenan en un *Red Black Tree*, que permite acceso a la tarea con menor *virtual runtime* en orden $O(log(n))$.

5.5.1. Compromisos

Para decidir la frecuencia de cambios de contexto se debe hacer un compromiso. Cambios de contexto más frecuentes mejoran la equidad pero degradan el rendimiento, por el contrario menos cambios llevan a menor equidad pero mejor rendimiento.

Este compromiso se gestiona mediante diversos parámetros de control, el más importante es `sched_latency`, es el tiempo total en que todas las tareas listas deberían ejecutarse al menos una vez. **CFS** divide este valor por la cantidad de procesos para obtener un `time_slice`, cuánto tiempo debe ejecutarse cada proceso, asegurando asignación de CPU equitativa. Si hay demasiados procesos en ejecución `time_slice` se volverá muy pequeño, causando demasiados cambios de contexto, para solucionar esto se le asigna una cota inferior a `time_slice`, se llama `min_granularity`.

Para tomar decisiones **CFS** es activado periódicamente mediante un temporizador. Cuando esto ocurre se determina si el proceso actual ha alcanzado el final de su ejecución. Dado que **CFS** lleva un seguimiento preciso del *virtual runtime* de todos los procesos, termina aproximando un reparto ideal de la CPU, incluso con los compromisos mencionados.

5.5.2. CFS y prioridades

El algoritmo descrito hasta el momento no considera las prioridades de las tareas. Para hacer esto **CFS** modifica el `time_slice` de cada proceso mediante un peso basado en la prioridad del mismo, a mayor peso mayor será el `time_slice`. Notar que la relación entre peso y prioridad no es lineal, sino logarítmica. El cálculo del *virtual runtime* es adaptado de una forma similar, se toma el tiempo real de ejecución del proceso y se lo escala de manera inversa según el peso de su prioridad.

5.5.3. Antes de CFS

El predecesor a **CFS** fue el **Scheduler O(1)**, añadido en la versión 2.4 del kernel, su principal ventaja era que la elección de la próxima tarea a ejecutar era $O(1)$. La idea es sencilla, se usa un arreglo de colas de tareas *activas* y otro de tareas *expiradas*. Cada vez que una tarea recibe su `time_slice` se transfiere al arreglo de tareas *expiradas*, una vez que el arreglo de tareas *activas* se vaciaba se intercambiaban los arreglos y se volvía a empezar. Elegir la próxima tarea era simplemente elegir el proceso en el arreglo *activo* en la cola de mayor prioridad.

En este sistema las tareas interactivas recibían un *priority bonus*, basado en un cálculo del tiempo que había pasado dormidas(`average sleep time`). Por el otro lado las tareas batch son penalizadas, decrementando su prioridad.

Dejó de utilizarse debido a que las heurísticas para manejar las tareas batch e interactivas se volvieron demasiado complejas y propensas a errores.

6. Administración de memoria

6.1. Funciones y operaciones

Los sistemas modernos poseen distintos tipos de memoria, con costos y velocidades distintas, que siguen la jerarquía Cache → Memoria principal(RAM) → Memoria secundaria(Almacenamiento). El procesador solo tiene acceso directo a los dos primeros tipos de memoria. Todo programa a ejecutar debe ser cargado en la memoria principal, siguiendo la arquitectura de Von Neumann. En un sistema moderno, multiusuario y multitarea, la memoria debe ser administrada para permitir que varios usuarios y procesos la compartan. Como hacer esto de forma eficiente y segura es el tema analizado en esta sección.

6.1.1. Espacio de direccionamiento

La arquitectura sobre la que corre un sistema determina dos factores importantes:

- **Tamaño de palabra:** La memoria de un sistema está estructurada como un arreglo direccionable de n bytes, es decir que al solicitar el contenido de una dirección el hardware entregará exactamente n bytes. El valor de n se llama *tamaño de palabra* de la arquitectura, por ejemplo `x86-64` tiene un tamaño de palabra de $n = 8$ bytes.
- **Espacio de direccionamiento:** Indica cual es la mayor dirección a la que puede hacer referencia la arquitectura, si esta tiene un *espacio de direccionamiento* de m bits entonces es posible referirse directamente a 2^m direcciones de memoria. Vale la pena notar que no es necesario que un sistema utilice todo el *espacio de direccionamiento* disponible; Usando como ejemplo nuevamente a `x86-64`, esta arquitectura podria direccionar hasta 18 446 744 073 709 551 616 bytes, sin embargo ningún procesador actual ofrece esta capacidad, los chips actuales están limitados a entre 2^{40} y 2^{48} bits.

6.1.2. MMU

Con al aparición de sistemas multitarea y multiusuario apareció la necesidad de tener más de un proceso cargado en memoria, a medida que los sistemas y programas crecieron en tamaño y complejidad se volvió necesario *abstraer* el espacio de almacenamiento, para dar la ilusión de contar con más memoria de la que realmente se tiene. La MMU es la encargada de esta tarea, al mismo tiempo se ocupa de verificar que un proceso no pueda leer o modificar los datos de otro, si el sistema tuviese que hacer esto sufriría una gran penalización en performance.

6.1.3. Caché

En esta sección hay un elemento que se asume como hecho, la memoria *cache*. Si bien es casi transparente para el sistema es muy importante mantenerla en mente. Como se vio anteriormente cuando un proceso se está ejecutando se producen accesos a memoria, ya sea para buscar datos o la próxima instrucción a ejecutar. Anteriormente esto no era un problema, sin embargo a lo largo del tiempo la velocidad de los procesadores ha sobrepasado con creces la de la memoria.

Esto es un problema grave, pues si el CPU solicita datos que aún no están disponibles este debe detener su ejecución(*stall*) hasta que estos estén disponibles. Todo instante donde se espera una transferencia de datos es tiempo de ejecución perdido.

La memoria *cache* intenta solucionar este problema, es una memoria de alta velocidad ubicada entre el CPU y la memoria principal. Guarda copias de los *recursos*[1] que van siendo accesados durante la ejecución, evitando así que ocurran *stalls*.

6.1.4. Localidad

El motivo por el que la memoria *cache* es (razonablemente) efectiva es el principio de *localidad de referencia*:

- **Localidad temporal:** Es probable que un recurso que fue utilizado recientemente vuelva a ser utilizado en un futuro cercano.
- **Localidad espacial:** La probabilidad de que un recurso aún no requerido eventualmente sea requerido es mucho mayor si está cerca de algún recurso que ha sido utilizado recientemente.
- **Localidad secuencial:** Un recurso, muy particularmente la memoria tiende a ser requerido de manera secuencial.

Aplicando este concepto cuando el CPU solicita una cierta dirección de memoria, también se transfieren los *alrededores* de la dirección solicitada al cache, hablando con propiedad estos *alrededores* se llaman bloque o página.

[1] Próximamente veremos qué forma tienen estos *recursos*.

6.1.5. Espacio en memoria de un proceso

Un proceso en ejecución posee una cierta estructura que le permite guardar la información relativa a su ejecución:

- **Sección de texto:** Es la imagen en memoria de las instrucciones a ser ejecutadas, generalmente ocupa las direcciones más bajas del espacio de memoria.
- **Sección de datos**: Espacio fijo preasignado para las variables globales y los datos pre inicializados. Su tamaño y ubicación es fijado al momento de la compilación y no puede cambiar, aunque sí pueden cambiar los valores almacenados en él.
- **Espacio de libres:** Espacio de memoria empleado para la asignación dinámica de memoria durante la ejecución del proceso. Se ubica por encima de la sección de datos y crece hacia arriba, es decir hacia direcciones más grandes. En inglés se llama *heap*.
- **Pila de llamadas:** Espacio de memoria empleado para almacenar la secuencia de funciones que han sido llamadas dentro del proceso, junto con sus parámetros, direcciones de retorno, variables globales, etc. Ocupa la parte más alta del espacio de memoria y crece hacia abajo, es decir hacia direcciones más chicas.

6.1.6. Resolución de direcciones

Ver si entra

6.2. Asignación de memoria contigua

Fue la primera solución implementada para lograr que múltiples procesos compartan memoria, se le asigna a cada programa a ser ejecutado un bloque contiguo de memoria de un tamaño fijo. Bajo este esquema el sistema operativo utiliza una región específica de memoria para sí mismo, generalmente la región baja. Una vez que el sistema tiene todo el espacio que necesita asigna el espacio restante a los procesos. Funciona siempre y cuando los programas permitan la resolución de direcciones en tiempo de carga o ejecución.

Existen tres estrategias que el sistema puede usar para asignar memoria a un proceso nuevo:

- **Primer ajuste:** El sistema toma el primer bloque de tamaño suficiente para alojar el proceso que encuentra. Es el más simple de implementar y de mas rápida ejecución, no obstante puede causar desperdicios de memoria.
- **Mejor ajuste:** El sistema busca entre todos los bloques disponibles cual es el que mejor se ajusta al tamaño requerido para alojar el proceso nuevo. Es menos eficiente que *primer ajuste*, pues implica recorrer toda la lista de bloques. Suele llevar a menos desperdicio de memoria.
- **Peor ajuste:** El opuesto a *mejor ajuste*, se utiliza el bloque más grande disponible. Es el más eficiente de implementar, si se usa una estructura de datos apropiada. Este método busca que los bloques libres luego de asignar memoria a un proceso sean lo más grandes posibles.

6.2.1. Fragmentación

Es un fenómeno que ocurre a medida que los procesos terminan su ejecución y el sistema libera la memoria asignada a ellos. Empiezan a aparecer regiones de memoria disponibles interrumpidas por regiones usadas por procesos activos, este fenómeno se conoce como *fragmentación*.

Vale la pena notar que existen dos tipos de fragmentación. La fragmentación *externa* ocurre cuando hay muchos bloques libres entre bloques asignados a procesos, la *interna* se refiere a la cantidad de memoria dentro de un bloque que nunca se usará. Este problema puede tener un efecto importante en el

uso de la memoria y por tanto el rendimiento del sistema, se estima que por cada N bloques asignados a un proceso se pierden aproximadamente $0{,}5N$ bloques por fragmentación *interna* y *externa*.

Un problema que puede surgir de este fenómeno es que el espacio total libre de memoria puede ser mayor a lo que requiere un proceso, pero al estar fragmentada en muchos bloques puede no haber uno lo suficientemente grande para cargarlo.

6.2.2. Compactacion

Una solución al problema mencionado en la sección anterior es la *compactación*, consiste en mover los contenidos de la memoria de tal forma que los bloques ocupados estén en espacios contiguos, unificando los bloques libres en uno solo. Si bien este método soluciona el problema de la fragmentación es costoso de hacer y requiere que los procesos utilicen resolución de direcciones al tiempo de ejecución.

6.2.3. SWAP

Es una técnica que puede ser utilizada por cualquiera de los esquemas de manejo de memoria presentados a continuación, le permite al sistema *comprometer*[2] más memoria de la que posee físicamente disponible. Cuando la memoria se acaba el sistema procede a copiar *secciones* de la memoria primaria a la memoria secundaria, lo que le permite liberar memoria primaria. Al utilizar esta técnica se debe considerar que la memoria secundaria, particularmente las unidades de disco son del orden de decenas de miles de veces más lentas que la memoria principal, por lo que este proceso es muy costoso de realizar.

En el caso de la asignación de memoria contigua las *secciones* son programas enteros.

6.3. Segmentacion

Es un concepto que aplica directamente sobre la arquitectura del CPU, cuando se compila para una arquitectura de este tipo las distintas secciones presentadas en la sección 6.1.5 se colocan en un segmento diferente. Cada uno de estos segmentos puede tener permisos diferentes, esto permite proteger los segmentos contra modificaciones y accesos inválidos. Este esquema sigue sufriendo problemas de segmentación externa, dado que al final del día se siguen asignando bloques de tamaño arbitrario en la memoria física.

Al implementar este tipo de esquemas los programas compilados utilizan *direcciones lógicas*, son una abstracción de las direcciones de memoria reales. En estos casos están compuestas de un selector de segmento y un desplazamiento dentro del segmento. Para interpretar esta dirección es necesario el uso de la `MMU`, utilizando una estructura auxiliar obtiene la dirección base del segmento, junto con su tamaño y atributos de protección; Luego se toma la dirección base del segmento y se le suma el desplazamiento para obtener la *dirección física*.

Vale la pena notar que este esquema permite realizar `SWAP` de forma más eficiente que su predecesor. Esto se debe a que se pueden enviar al disco cada uno de los segmentos por separado. Al mismo tiempo si un segmento es solo de lectura solo es necesario copiarlo al disco una única vez, pues sus contenidos no pueden ser modificados.

6.3.1. Excepciones

La traducción de una *dirección lógica* a una *dirección física* puede fallar por diferentes razones, cuando esto ocurre el sistema operativo genera una excepción. En caso de haber más de una excepción el sistema reacciona primero a la más severa. Las más comunes son:

- **Violación de segmento:** El desplazamiento indicado es mayor al tamaño definido para el segmento.

[2] Es decir asignar

- **Violación de seguridad:** El tipo de acceso pedido no coincide con los permisos del segmento.
- **Segmento no presente:** El segmento indicado no se encuentra en memoria.

6.4. Paginacion

Los problemas mencionados en las secciones anteriores sobre fragmentación externa pueden solucionarse mediante el uso de paginación. Se basa en dividir la memoria física y el espacio de direcciones para cada proceso en una serie de marcos y páginas, respectivamente, todas del mismo tamaño. La `MMU` se encarga del mapeo entre páginas y marcos mediante la *tabla de páginas*, transformando una *dirección lógica* en una *dirección física*. Notamos que cada proceso tiene su propia tabla de paginas, esta se almacena en el `PCB`.

Bajo este esquema la dirección lógica está compuesta de un *identificador de página* y un *desplazamiento*. Esto es similar al esquema usado por *Segmentación*, con la gran diferencia de que en este caso los segmentos son de longitud fija. De la misma forma que bajo *Segmentación* cada pagina tiene permisos, indicando que operaciones pueden realizarse sobre ella.

Es importante notar que es posible que una *página virtual* no se encuentre en la memoria principal, es decir que esta no tiene una *dirección física*. En este caso ocurre un *fallo de página*, el sistema operativo debe manejar esta excepción asignándole un marco de la memoria a la página. Este esquema de asignación se ve en detalle en la sección de *memoria virtual*.

6.4.1. Tamaño de los marcos

El tamaño de los marcos (y, por lo tanto, las páginas) debe ser una potencia de dos, de modo que la `MMU` pueda obtener facilmente el *identificador de página* y el *desplazamiento*, mediante shifts de bits. Este varía entre 512 *bytes* (2^9) y 16 MB (2^{24}).

Ahora, si bien la fragmentación externa puede solucionarse usando paginación, el problema de la fragmentación interna persiste. Al dividir la memoria en bloques de longitud fija de 2^n *bytes*, en promedio un proceso desperdiciara $\frac{2^n}{2}$ *bytes*.

Esto lleva a pensar que las páginas deberían ser lo más pequeñas posibles, sin embargo esto lleva a una gran sobrecarga administrativa, que se vuelve un limitante en el sentido opuesto. Por una lado las transferencias entre unidades del disco y memoria son mucho más eficientes si pueden mantenerse como recorridos continuos. Al mismo tiempo el `PCB` incluye la información de memoria del proceso, a mas paginas, mas grande se vuelve el `PCB`, lo que implica más información a intercambiar durante un cambio de contexto. Estas limitaciones apuntan a que se debe mantener un tamaño de página más grande, aceptando algo de fragmentación interna.

6.4.2. La tabla de páginas y el TLB

En un primer momento la tabla de páginas era tan pequeña que podía almacenarse en un conjunto especial de registros. En los sistemas actuales, incluso con un tamaño de página *muy grande*, sería imposible, pues los requerimientos de memoria siguen siendo demasiado grandes.

La solución actual es almacenar la tabla de páginas en la memoria principal y apuntar a esta con un par de registros especiales. Si bien este esquema tiene un coste muy bajo y permite cambios de contexto rápidos tiene un problema importante, la velocidad. Cada acceso a memoria requiere de dos accesos, uno a la tabla de páginas para resolver la dirección y otro para acceder a la memoria en sí.

La solución obvia es usar un caché, sin embargo este no es un cache generico, la `MMU` utiliza un cache especial llamado `TLB`(*Translation lookaside buffer*). Es una tabla hash, almacenada en memoria de alta velocidad, donde las llaves son las páginas y los valores los marcos correspondientes, lo que permite realizar traducciones en tiempo constante.

Cuando el CPU realiza un acceso a memoria, si la página solicitada está en la `TLB` la `MMU` tiene su *dirección física* de inmediato. En caso contrario se genera un *fallo de página*, donde se consulta a la tabla de páginas(en memoria principal), la *dirección física* correspondiente. Esta entrada es luego

añadida a la TLB. Gracias a las propiedades de *localidad de referencia*, la probabilidad de que las regiones más empleadas de memoria no generen fallos de la TLB son muy altas, aun si la TLB tiene pocas entradas.

Dado que el tamaño de la TLB es limitado es necesario implementar una política de reemplazo, que indique que entrada debe ser desalojada cuando la TLB está llena. Uno de los esquemas más comunes es LRU(*Least recently used*), mas adelante se explicaran distintos esquemas de desalojo en detalle.

6.4.3. El tamaño de la tabla de páginas

Incluso utilizando un TLB el espacio utilizado por las tablas de páginas sigue siendo demasiado grande, hay dos soluciones posibles:

- **Paginación multinivel:** Aprovechando que la mayor parte del espacio de direccionamiento de un proceso suele estar vacío(el *stack* y *heap*), se puede subdividir el *identificador de página* en dos o más niveles, disminuyendo la cantidad de memoria usada. Sin embargo se debe considerar que cada nivel de paginas adicional conlleva un acceso adicional a memoria en caso de un fallo de página.
- **Tabla hash:** Se mapea cada una de las tablas páginas a un espacio muestral mucho más pequeño, esto hace que el tamaño de la tabla de páginas sea dinámico, crece según el uso de memoria del proceso. Si se usa este método debe elegirse una función de hash muy eficiente, para evitar que el tiempo de que toma calcular la posición de un elemento en la tabla resulte significativo.

6.5. Memoria compartida

Hay situaciones en las que diferentes procesos pueden beneficiarse de compartir áreas de memoria. Una de ellas es como método de IPC(*Inter process comunication*), en que dos o más procesos intercambian estructuras de datos complejas sin incurrir el costo de copiado.

Otro caso mucho más frecuente es compartir código, si un mismo programa es ejecutado muchas veces y este no posee código automodificable, no tiene sentido que las páginas que representan cada instancia están asociadas a marcos diferentes. En estos casos el sistema operativo puede asignar los mismos marcos a páginas de distintos procesos. Esto es particularmente útil para las bibliotecas del sistema, que son utilizadas por una gran cantidad de programas.

6.5.1. Copy on write

En los sistemas Unix el mecanismo más utilizado para crear procesos nuevos es la llamada al sistema `fork()`. Cuando se utiliza, el sistema operativo crea un proceso idéntico al que llamó a la función, la única diferencia es el valor de retorno de `fork()`.

En un principio todas las páginas de memoria del proceso padre se comparten con el proceso hijo, evitando el gran costo de realizar una copia. Sin embargo si uno de los procesos modifica un dato en una de estas páginas esta se copia a un nuevo marco y deja de ser una página compartida.

Esto se puede lograr marcando todas las páginas compartidas como solo de lectura, luego cuando uno de los procesos intente modificar alguna de estas páginas se generará un fallo. Al notar que esto ocurre sobre un espacio *copy on write* el sistema operativo responde copiando la página en cuestión y marcándola como de lectura y escritura.

6.6. Memoria virtual

Varios de los aspectos mencionados en las secciones anteriores conforman lo que se conoce como *memoria virtual*. Los procesos trabajan con una idealización de la memoria, en la cual ocupan el espacio completo de direcciones, por lo que los procesos no conocen su dirección en memoria relativa a otros procesos, esto es manejado de forma transparente por el sistema.

Este esquema es particularmente útil pues le permite al sistema comprometer más memoria de la que realmente tiene, como se vio anteriormente esto se hace mediante una técnica llamada `SWAP`, al mismo tiempo permite cargar procesos de forma más eficiente y rápida. Las secciones a continuación explican estos dos puntos en más detalle.

6.6.1. Paginacion sobre demanda

La memoria virtual entra en juego desde la carga misma del proceso. Si bien en una computadora con arquitectura de Von Neumann es imposible ejecutar código que no está en la memoria principal, si se puede comenzar a ejecutar un programa sin que *todo* su código está cargado. Esto es particularmente útil ya que la mayoría de programas tienen código que solo se emplea eventualmente(si es que se emplea), por lo tanto no tiene sentido cargar todo el programa en memoria.

La *paginación sobre demanda* implica que, para comenzar a ejecutar un proceso, el sistema operativo solo carga en la memoria principal la porción necesaria del programa para iniciar su ejecución. Bajo este esquema el paginador es flojo, solo carga páginas en la memoria principal si es requerido.

Para implementar esto se le añade un *bit de validez* a todas las entradas de la tabla de páginas, que indica si esta se encuentra en la memoria principal. Si el proceso intenta acceder a una página que no se encuentra en memoria esto causa un *fallo de página*, este es manejado de la siguiente forma:

1. Se verifica en el `PCB` si la solicitud corresponde a una página que ya ha sido asignada a este proceso.
2. En caso de que la referencia sea inválida, se termina el proceso.
3. Se busca un marco disponible para ubicar la página, puede ser necesario elegir un marco víctima para desalojar.
4. Se carga la página al marco correspondiente.
5. Una vez finalizada la carga se actualiza el `TLB` y `PCB` indicando que la pagina esta en memoria.
6. El proceso continúa su ejecución, ejecutando la instrucción que desencadenó el fallo, todo este proceso es transparente para el proceso.

6.6.2. Reemplazo de páginas

Como se mencionó al comienzo de esta sección es posible que el sistema operativo comprometa más memoria de la que físicamente tiene, cuando esto ocurre el sistema puede quedarse sin memoria primaria física. Esto es un problema grave, pues uno de los objetivos de un sistema operativo es otorgar a usuarios y procesos la ilusión de que son los únicos en el sistema. Por lo tanto no sería aceptable terminar la ejecución de un proceso ya aceptado para liberar memoria.

Para solucionar este problema se emplea un esquema llamado *reemplazo de páginas* o `SWAP`. El mecanismo básico es simple, si se requiere cargar una pagina pero todos los marcos de la memoria principal están ocupados, el sistema elige una *pagina víctima* que será almacenada en el espacio de *intercambio* o `SWAP` del disco. Luego este espacio liberado puede ser usado para cargar la página deseada.

Vale la pena notar que este proceso es ineficiente, pues implica una doble transferencia al disco, una para almacenar la página víctima y otra para leer la página cargada. Por lo tanto es deseable minimizar la cantidad de intercambios realizados, para esto se utilizan diversos algoritmos, que se analizaron a continuación así como un entrada adicional a cada página en la tabla de páginas.

Esta entrada es un bit, llamado bit de modificación(*dirty*), se marca como apagado cuando la página se carga en la memoria principal. El hardware lo enciende automáticamente cuando hay una escritura a la página. Cuando se elige una página víctima se verifica su bit de referencia, si está apagado se sabe que la información sobre la página en el disco sigue siendo válida, por lo que no es necesario guardarla en el disco, evitando una transferencia.

6.6.3. Anomalía de Belady

Un fenómeno interesante que se presenta en algunos de los algoritmos explicados a continuación es la *anomalía de Belady*. Contrario a la lógica algunos algoritmos de reemplazo, con ciertas secuencias de acceso experimentan una degradación en su performance conforme aumenta la cantidad de marcos de memoria disponibles. Es importante notar que si bien esta anomalía es importante a la hora de analizar algoritmos de reemplazo su efecto en sistemas reales es casi nulo.

6.6.4. FIFO(First in first out)

El algoritmo más simple posible, cada vez que se carga un marco en memoria se toma nota del tiempo. Cuando es necesaria una víctima se elige el marco que haya sido cargado hace más tiempo. Notas:

- Muy fácil de implementar, con una lista enlazada circular.
- No considera las últimas solicitudes.
- Todos los marcos tienen la misma probabilidad de ser reemplazados.
- Vulnerable a la anomalia de Belady.

6.6.5. OPT, MIN(Optimal, Minimal)

Algoritmo de interés puramente teórico, se elige como víctima al marco que no va a ser utilizado por un tiempo máximo, o nunca más. No es implementable en la práctica porque requiere conocer el comportamiento futuro del sistema. Notas:

- Ofrece una cota mínima sobre la cantidad de fallos de paginas, util para evaluar a otros algoritmos.
- Libre de la anomalia de Bealdy.

6.6.6. LRU(Least recently used)

Busca acercarse a *OPT*, intenta predecir cuándo será la próxima vez que se necesite cada marco basado en su historia reciente. Elige como víctima al marco que no ha sido utilizado hace más tiempo. Su implementación es difícil y requiere apoyo de hardware. Notas:

- Libre de la anomalia de Belady.
- Una implementación posible es agregar un contador de acceso a cada marco, se actualiza cada vez que se hace referencia a este. Luego se elige como víctima el marco cuyo contador es menor.
- Generalmente es mejor que *FIFO*.

6.6.7. MFU, LFU (Most frequently used, least frequently used)

Ambos algoritmos se basan en mantener un contador, como *LRU*, pero en vez de tiempo miden la cantidad de referencias a una página. Cuando se necesita una víctima *MFU* elige el marco con mayor contador, mientras que *LRU* el marco con el menor contador. Son casi tan caros de implementar como *LRU*, con peor rendimiento, casi no se usan.

6.6.8. Aproximaciones a LRU

Dada la complejidad de implementar *LRU*, los siguientes algoritmos buscan ser una aproximación de este:

- **Bit de referencia:** Una aproximación simple. Consiste en que todas las entradas de la tabla de marcos tengan un bit adicional, llamado de referencia(*use*), este permite distinguir los marcos usados recientemente. Al comenzar la ejecución todos los bits están apagados, cada vez que se referencia un marco su bit se enciende, periódicamente el sistema operativo apaga todos. Cuando se debe elegir una víctima se usa *FIFO* sobre todos los marcos cuyo bit de referencia está apagado.
- **Columna de referencia:** Mejora trivial sobre el esquema anterior, cada marco tiene varios bits de referencia. Cada vez que el sistema operativo anula los bits de referencia el valor anterior es almacenado en la columna de referencia. Cuando el sistema debe elegir una víctima lo hace del conjunto cuya columna de referencia tiene el valor más bajo.
- **Segunda oportunidad:** También utiliza un bit de referencia y un recorrido tipo *FIFO*. La diferencia es que el bit de referencia puede ser apagado por el algoritmo. Mantiene un puntero a la próxima víctima, cuando ésta es requerida se verifica si su bit de referencia es cero, si es así el marco es seleccionado. De lo contrario el bit se apaga y el puntero avanza, dando al marco una segunda oportunidad de ser elegido. En el peor caso degenera en *FIFO*.
- **Segunda oportunidad mejorada:** Utiliza un esquema similar al algoritmo anterior, pero considerando el bit de modificación, elige una víctima de acuerdo al siguiente orden de preferencia:

  1. **(0, 0)** El marco no ha sido utilizado ni modificado recientemente, un candidato ideal para reemplazo.
  2. **(0, 1)** El marco no ha sido utilizado recientemente pero esta modificado, no es tan buena opción pues se debe guardar la página en el disco antes de reemplazarla.
  3. **(1, 0)** El marco no está modificado pero fue utilizado recientemente, debido a la localidad temporal es probable que vuelva a ser utilizado. No es una buena opción.
  4. **(1, 1)** El marco fue utilizado recientemente y está modificado, la página debe ser escrita en el disco y es probable que vuelva a ser utilizado. La peor opción, se debe evitar reemplazarla.

  Vale la pena notar que el algoritmo no realiza exactamente el "recorrido" mostrado, en lugar de eso al realizar la segunda pasada apaga el bit de referencia, y luego realiza dos pasadas más; Es decir que la tercera y cuarta pasada no ocurren nunca, en su lugar hay dos pares de las dos primeras pasadas.

En conclusión todas las aproximaciones a *LRU* se basan en buscar marcos que no fueron referenciados recientemente ni modificados, pues son las mejores víctimas.

6.7. Asignación de marcos

Abordando el problema desde el lado opuesto vale la pena considerar ¿Cómo se asignan los limitados marcos de la memoria principal a los distintos procesos del sistema? En principio observamos que la paginación exclusivamente por demanda no es eficiente, pues se generan demasiados fallos de pagina. Es preferible que el sistema sea un poco *ansioso* y asigne desde un principio un número mínimo utilizable de marcos.

Esto nos lleva a considerar cuánto es el mínimo de marcos que un proceso necesita, naturalmente si un proceso tiene asignados muy pocos marcos su rendimiento se verá afectado. En un principio esto significa que todo proceso requiere al menos dos marcos de memoria, para manejar accesos a memoria directos, particularmente situaciones donde la dirección deseada no está en el marco actual. Sin embargo, esto no es suficiente en una arquitectura moderna, cada instrucción puede desencadenar varios fallos de página, a causa de la posibilidad de realizar referencias indirectas a memoria. A grandes rasgos cada nivel de indirección permitido aumenta en uno la cantidad mínima de marcos necesaria.

6.7.1. Esquemas de asignación

Habiendo establecido el número mínimo de marcos de un proceso debemos entonces determinar el número deseable, hay dos grandes estrategias para hacer esto:

- **Asignación igualitaria:** Se divide el total de la memoria física de forma igualitaria entre todos los procesos en ejecución. Si bien este esquema es muy simple, también es muy ineficiente, naturalmente distintos procesos tienen requerimientos de memoria distintos. Por lo tanto este esquema tiene grandes problemas de sobre y sub asignación de memoria.
- **Asignación proporcional:** A cada proceso se le asigna un porción de la memoria física de acuerdo a su uso de memoria virtual. Cabe notar que este esquema debe respetar ciertos límites, independientemente del uso de memoria virtual. Por un lado nunca debe asignar menos del número mínimo de marcos a un proceso. Por el otro lado debe cuidar no sobreasignar recursos a un proceso *obeso*, es decir uno mucho mas grande a todos los demas, pues esto los penalizaria.

  Este esquema sufre particularmente cuando cambia el nivel de multiprogramación, pues es necesario recalcular la proporción de memoria correspondiente a cada proceso. Tampoco considera el hecho de que muchos procesos tienen un patrón irregular de uso de recursos, lo que lleva a situaciones de sobre y sub asignación de memoria, generalmente de menor gravedad que las causadas al usar *asignación igualitaria*.

6.7.2. Ámbitos del algoritmo de reemplazo de páginas

Para atender los problemas no resueltos que se describieron en la sección anterior, se puede considerar el entorno en el que opera el algoritmo de reemplazo de páginas:

- **Reemplazo local:** Ocurre al nivel de cada proceso, trata de mantener la asignación hecha por el esquema de asignación global. Cuando ocurre un fallo de página los únicos marcos considerados como víctima son los que pertenecen al mismo proceso.
- **Reemplazo global:** El algoritmo de asignación determina el espacio asignado a un proceso cuando este es inicializado, e influyen sobre los algoritmos de reemplazo. Operan sobre el espacio completo de memoria, los algoritmos vistos anteriormente son de este tipo.
- **Reemplazo global con prioridad:** Es un esquema mixto, en él un proceso puede sobrepasar su límite de marcos siempre que le quite marcos a un proceso de menor prioridad. Es consistente con el comportamiento de los schedulers, dándole preferencia a procesos de mayor prioridad.

6.8. Hiperpaginación

Es un fenómeno que puede ocurrir por distintos motivos, e implica una alta tasa de la actividad de paginación. Causa que la mayor parte del tiempo de ejecución sea el sistema lidiando con fallos de página y SWAPs, en lugar de realizando trabajo. Esto causa que el rendimiento global del sistema disminuya, afectando a todos los procesos.

Bajo un esquema local puede ocurrir cuando un proceso tiene asignadas pocas páginas para llevar a cabo su trabajo, generando fallos de página constantes. En un esquema global puede ocurrir cuando hay demasiados procesos en ejecución y el sistema sobrecompromete demasiada memoria.

Una forma que tiene el sistema operativo de lidiar con este problema es disminuyendo el nivel de multiprogramación. Se selecciona uno(o más) procesos y se los suspende por completo, quitándoles toda su memoria física, liberando así recursos para el resto del sistema. Los procesos seleccionados pueden ser los de menor prioridad, que estén utilizando más marcos o hayan causado más fallos de página.

7. Memoria virtual en Linux

7.1. Organización de la memoria

Como se vio anteriormente todo sistema operativo debe decidir dónde ubicarse a sí mismo en memoria respecto a los procesos. En Linux, si bien la distribución exacta depende de la arquitectura del hardware vemos el caso más común, `x86-64`:

- Bloque `[0 .. 0x0fff]` (Primeros 4kb): Reservado para el hardware.
- Bloque `[0x1000 .. 0x9ffff]`: Disponible para memoria dinamica.
- Bloque `[0xa0000 .. 0xfffff]`: Reservado para el hardware, usado por dispositivos mapeados a memoria y rutinas de la BIOS.
- Bloque `[0x100000 .. 0x100000 + sizeof(kernel)]`: Núcleo del sistema.
- Bloque `[0x100000 + sizeof(kernel) .. DIRMAX]`: Disponible para memória dinámica.

Lo importante de esta distribución es que el núcleo se carga después de la memoria reservada para el hardware para poder cargarlo en un espacio de memoria contiguo, de esta forma el código del núcleo se mantiene sencillo y más fácil de cargar.

Vemos ahora otras características de como Linux gestiona la memoria:

- **Memoria compartida:** Una página en memoria puede ser referenciada por dos o más tablas de paginación. Además se implementa *Copy on write* para evitar duplicar páginas entre un proceso padre y sus hijos.
- **Mapeo de archivos a memoria:** Un proceso puede solicitar que una parte de su memoria virtual este mapeada a un archivo. De esta forma, desde el punto de vista del proceso, interactuar con el archivo se reduce a leer/escribir en una dirección de memoria.
- **Lectura por adelantado:** Cuando se lee un bloque de un disco mecánico la probabilidad de que sea necesario leer el siguiente bloque en poco tiempo es muy alta. Por lo tanto el sistema lee uno(o más) bloques siguientes, y los almacena en un buffer. Si el proceso los requiere esto disminuye mucho el tiempo de acceso.
- **NUMA:** El acceso no uniforme a memoria tiende a usarse en equipos multiprocesadores. Permite que ciertas porciones de la memoria estén asociadas a un procesador, dando a este una mejor velocidad de acceso. Vale la pena notar que el acceso por parte de los demás procesadores no está restringido, simplemente será más lento.
- **Reverse mapping:** El algoritmo de reemplazo de páginas necesita información adicional sobre los marcos físicos. Para acceder a esta información de forma eficiente se usa mapeo inverso.
- **Reservas de memoria:** Hay ciertos pedidos de memoria que requieren tratamiento prioritario, por ejemplo durante el manejo de una interrupción. Para esto Linux mantiene una pool de memoria especial, destinadas a estos casos; Se accede a esta mediante un pedido de ubicación atómica, que resuelve la demanda inmediatamente o retorna un error.
- **Asignación a cachés:** La RAM se asigna indistintamente a procesos y a caché, todos pueden crecer indiscriminadamente.

7.2. Ubicación de memoria y gestión de memoria libre

Si bien Linux utiliza paginación en los espacios de direcciones de procesos, hacer esto para la memoria gestionada por el núcleo resulta casi imposible. Esto se debe a que los controladores `DMA`, utilizados por dispositivos de `I/O` requieren memoria contigua para leer y escribir datos de forma masiva. Al mismo tiempo cada vez que se modifican las tablas de paginación deben invalidar los contenidos de la *TLB*; Esto ocurriría de forma demasiado frecuente en el núcleo, penalizando el rendimiento.

Para solucionar este problema Linux utiliza dos estrategias:

- **Buddy system:** Es un sistema de alocación de memoria que se basa en dividir la memoria física en particiones, bajo este cada bloque de memoria tiene un tamaño que es potencia de $2(8KB,\ 16KB,\ 32KB,\ ...)$. Cuando se realiza un pedido de memoria la cantidad se redondea a la potencia de dos mas cercana($14KB$ se redondea a $16KB$), si no hay un bloque del tamaño adecuado se divide un bloque más grande en dos *buddies* de igual tamaño(Un bloque de $32KB$ en dos de $16KB$). Cuando un bloque de memoria se libera el sistema revisa si su *buddy* también está libre, si es así los combina para formar un solo bloque de mayor tamaño.

  Toda esta información se almacena en una lista de listas, donde cada sublista almacena los bloques de un tamaño. El índice 0 contiene los bloques de menor tamaño, es decir 1 bloque. Este sistema limita la fragmentación externa y permite alocar y liberar memoria muy rápidamente.

- **Slab allocator:** El *Buddy system* tiene una limitación importante, el tamaño mínimo que puede manejar es una página, Para reservar porciones de memoria más chicas, algo usual para el kernel, se usa el *Slab allocator*. Utiliza caches compuestos de *slabs*, bloques contiguos de memoria.

7.3. Reclamo de páginas

Un poco antes de que la memoria principal se llene el algoritmo de reclamo de páginas(*PFRA*) se encarga de incrementar la cantidad de bloques libres, quitándole marcos a procesos modo usuario y caches del núcleo, manteniendo así un pool de marcos libres mínimo. Esto se debe hacer antes de que la memoria se acabe, pues para escribir datos al disco(Si es necesario) se necesitan algunos marcos libres. El sistema hace esto de forma periódica mediante el proceso `kswapd`.

Este algoritmo maneja los marcos de forma distinta según su contenido:

- **No reclamable:** Paginas libres o reservadas, memoria dinamica o stack del kernel, paginas bloqueadas.
- **Swapeable:** Anónimas en espacio de direcciones modo usuario[3], páginas para *IPC*.
- **Sincronizable:** Cache y datos de archivos de disco, buffers de dispositivos, caches de disco.
- **Descartable:** No usados del caché de memoria o entradas del sistema de archivos.

Vale la pena notar que *PFRA* no sigue ninguna política en particular, sino que tiene en cuenta una serie de criterios a la hora de elegir que marcos liberar:

- Liberar primero las páginas que provoquen menos daño, por ejemplo caches.
- Todas las paginas no bloqueadas de un proceso son reclamables.
- Antes de liberar una página compartida se deben ajustar las tablas de paginación de todos los procesos involucrados.
- Reclamar solo páginas no utilizadas recientemente.
- En lo posible elegir páginas no modificadas.

[3]Son páginas que forman parte del espacio de memoria virtual de un procesos y no contienen datos provenientes de archivos.

7.4. OOM Killer

A pesar de los métodos de manejo y reclamo de memoria antes mencionados es posible que la cantidad de memoria libre del sistema se vuelva peligrosamente baja. Si esto ocurre, en un primer momento el sistema disminuye el tamaño de los caches.

Si la situación sigue empeorando y el sistema se queda sin `SWAP` para realizar las operaciones normales el núcleo puede decidir terminar un proceso sin enviarlo a `SWAP`, este proceso es elegido por el *OOM Killer*. Vale la pena notar que esta decisión no es sencilla, se intenta elegir un proceso bastante grande, que no haya hecho demasiado progreso y no esté involucrado en secciones críticas o tareas del núcleo.

7.5. `SWAP` en Linux

En Linux el sistema de `SWAP` va más allá de simplemente enviar paginas al disco:

- **Cache de `SWAP`:** Cuando se escribe una página en el disco también se la almacena temporalmente en un cache. Si la página es requerida poco tiempo después de haber sido puesta en `SWAP` se la puede buscar del caché, evitando tener que leer el disco.
- **Multiple `SWAP`-in:** El kernel debe evitar que una página compartida que está en el disco termine dos veces en memoria, para esto el núcleo utiliza primitivas de sincronización.
- **`SWAP-in` y `SWAP-out` concurrentes:** Se deben sincronizar correctamente las operaciones de `SWAP` para evitar conflictos. Si se envía una página a `SWAP` y el proceso intenta acceder a esta antes de que se termine de escribir al disco el sistema puede detectar que no está en memoria y traer una versión antigua.
- **`SWAP` a `RAM` o `zram`:** Otra forma de `SWAP` soportada por Linux se basa en crear un disco virtual en la `RAM` y enviar alli páginas comprimidas. La idea detrás de este esquema es mantener los beneficios del `SWAP` tradicional sin incurrir las pérdidas de rendimiento usuales. Es importante notar que este esquema convive con el `SWAP` tradicional, no es un reemplazo, sino otra alternativa antes de tener que recurrir a la memoria secundaria.

8. Organización de archivos

8.1. Conceptos básicos

Una de las principales abstracciones que provee un sistema operativo moderno es el *sistema de archivos*, la posibilidad de almacenar datos en *archivos* de forma ordenada en un *directorio jerárquico*. A lo largo de esta sección veremos cómo se estructura este modelo y que formas hay de implementar tal abstracción.

8.1.1. Concepto de archivo

A grandes rasgos un archivo es un tipo de datos abstracto, visto desde el punto de vista de un proceso solo es posible interactuar con ellos mediante la interfaz ofrecida por el sistema operativo[4]. Para el usuario los archivos son la mínima unidad lógica de almacenamiento, todo el almacenamiento persistente en el sistema se efectúa dentro de archivos, de forma directa e indirecta. Dentro de cada volumen o medio de almacenamiento los archivos disponibles conforman un directorio. Normalmente se los identifica mediante un nombre o ruta.

Cada sistema define la interfaz de archivos de acuerdo a sus objetivos, pero en lineas generales, siempre están disponibles las operaciones: `Borrar`, `Abrir`, `Cerrar`, `Leer`, `Escribir`, `Reposicionar`[5].

[4] Se verá más adelante que esto no es *del todo* cierto.

[5] La lectura y escritura a archivos se hacen siguiendo un puntero, reposicionarlo permite trabajar sobre distintas partes del archivo.

Siguiendo esta línea, la mayoría de sistemas proveen otra abstracción, el *descriptor de archivo*, es un identificador utilizado por los procesos a la hora de interactuar con un archivo, se obtiene al abrir el archivo, es provisto por el sistema.

8.1.2. Tabla de archivos abiertos

Tanto el sistema operativo como cada uno de los procesos mantiene una *tabla de archivos abiertos*. Estas mantienen información sobre todos los archivos abiertos, identificados mediante el *descriptor de archivo*.

El sistema y los procesos mantienen tablas separadas ya que cada uno maneja información distinta. A grandes rasgos el sistema necesita más información de bajo nivel sobre el archivo(Ubicación en el disco, información de bloqueo, etc), mientras que a el usuario solo le interesa la información de alto nivel(Permisos, nombre, *descriptor de archivo*, etc).

8.2. Acceso concurrente

En los sistemas multitarea puede ocurrir que dos(o más) procesos quieran acceder a un archivo en el mismo momento. En estos casos el acceso se clasifica de dos formas:

- **Acceso compartido vs exclusivo:** Varios archivos pueden obtener un *lock* de lectura sobre el mismo archivo, garantizando que sus contenidos no sufrirán cambios. Por el otro lado si un proceso obtiene un *lock* exclusivo sobre un archivo indica que este va a ser modificado, naturalmente solo un proceso puede tener un *lock* exclusivo.
- **Bloqueo obligatorio vs consultivo:** En el primer caso cuando un proceso adquiere un *lock* el sistema operativo se encarga de imponer las restricciones apropiadas a todos los demás procesos, independientemente de si fueron programados para considerar esta posibilidad. En contraste, en un sistema consultivo, los *locks* se manejan de forma cooperativa entre todos los procesos involucrados, respetar el *lock* depende de cada proceso.

Vale la pena notar que no todos los sistemas operativos implementan las cuatro combinaciones posibles. Como regla general los sistemas Windows utilizan un esquema de bloqueo obligatorio y los sistemas Unix bloqueo consultivo.

8.3. Tipos de archivo

Naturalmente hay distintos tipos de archivos, surge entonces la pregunta de como diferencia el sistema los distintos tipos de archivos. Hay tres estrategias principales para hacer esto:

- **Extensión:** El nombre de cada archivo se divide en dos partes mediante un punto(`.`), el nombre del archivo en sí y su extensión. En estos casos el sistema mantiene una lista de extensiones conocidas y cómo actuar frente a cada una de ellas. El problema de este esquema es que es posible esconder la extensión de un archivo, permitiendo que un actor malicioso esconda la verdadera naturaleza de un archivo. (Usado por Windows)
- **Números mágicos:** El tipo de un archivo se reconoce mediante los primeros bytes, llamados *números mágicos*. Cuando no es posible hacer este reconocimiento se usan heurísticas para tratar de determinar el tipo de archivo. Un tipo de *número mágico* especial es el *hashbang*(`#!`), en sistemas Unix indica que el archivo es un *script*. (Usado por Unix)
- **Metadatos externos:** En este esquema la información de un archivos se separa en dos divisiones(*forks*), los datos que constituyen el archivo en si son la *division de datos* y la información sobre el archivo constituye la *division de recursos*. Esta segunda division puede contener una gran cantidad de información, incluyendo el programa generó el archivo, dando su tipo. (Usado por MacOS)

8.4. Estructura

La mayor parte de los sistemas operativos trabaja con archivos sin estructura, el sistema trata a cada archivo como una secuencia de bytes, las aplicaciones son las responsables de preparar la información de forma congruente. Esto se debe a que imponer una estructura a los archivos resta flexibilidad al sistema y crea demasiado acoplamiento entre el sistema y las aplicaciones.

Vale la pena notar que los descendientes de *MS-DOS*, en particular *Windows*, poseen un remanente de los archivos estructurados, un archivo puede ser *de texto* o *binario*.

8.5. Métodos de acceso

El acceso a los archivos puede realizarse de distintas formas dependiendo el sistema y los requisitos del programa:

- **Secuencial:** La forma de acceso más sencilla, se leen o escriben bytes de forma consecutiva hasta que se llega al final del archivo o los datos. Si bien es adecuado para datos lineales(código, documentos, imágenes, etc) leer datos estructurados(bases de datos) de esta forma sería muy ineficiente.
- **Aleatorio:** En este caso el archivo se considera como una secuencia numerada de registros, y no existe restricción alguna sobre el orden de acceso.
- **Relativo a índice:** Se utiliza para archivos *débilmente* estructurados, con registros de tamaño variable. Para implementar este tipo de acceso el conjunto de datos se divide en dos secciones, la primera almacena el punto de inicio y fin de cada uno de los registros, la segunda almacena los datos propiamente dichos. Para acceder a un registro se utiliza acceso aleatorio sobre la porción correspondiente de la sección de datos.

8.6. Archivos especiales

Dada la facilidad de manejo y comprensión de los archivos los sistemas operativos los aprovechan para representar todo tipo de estructuras, no solo archivos. En este sentido en Unix todo es un archivo, más específicamente un archivo puede pertenecer a las siguientes categorías:

- **Archivo estándar:** El tipo básico de un archivo, representa un archivo propiamente dicho.
- **Objetos del sistema de archivos:** La información del sistema de archivos se almacena dentro de archivos, aunque estos reciben tratamiento especial. En estos casos el archivo *real* es ocultado por el sistema operativo, que gestiona la información que este posee para que el usuario pueda usarla.
- **Dispositivos:** Permiten el acceso directo a un dispositivo externo, pueden ser de *bloques* o *caracteres*. El contenido de un archivo de este tipo son dos números, el *mayor* y el *menor*, el primero indica de qué clase de dispositivo se trata y el segundo de qué instancia.
- **Comunicación entre procesos:** Los archivos pueden representar *tuberías nombradas* y *sockets*, ambos mecanismos de comunicación entre procesos.

8.7. Directorios

Una interpretación de alto nivel de un directorio, es la de una tabla de símbolos, que traduce nombres de archivos a entradas en el directorio, es decir los archivos en sí. Veremos que el concepto de directorio ha evolucionado a lo largo del tiempo, para responder a los cambios en las necesidades de los sistemas y usuarios.

Vale la pena notar que el carácter utilizado para separar los elementos de un directorio no es estándar. Los más comunes hoy en dia son la diagonal(/) utilizada por sistemas Unix y la diagonal invertida(\) utilizada por sistemas Windows. Utilizaremos la primera(/) a lo largo de esta sección.

Al mismo tiempo el directorio determina el formato que debe seguir el nombre de cada archivo(y por lo tanto directorio). En un sistema moderno es común que el nombre de un archivo pueda tener hasta 256 caracteres, incluyendo espacios, caracteres especiales, etc.

8.7.1. Sistema de archivos plano

Los primeros sistemas limitaban el concepto de directorio a una lista de los archivos que lo conformaban, no existía el concepto de jerarquía. Esto era aceptable pues en aquellos tiempos los archivos no permanecían demasiado tiempo en el disco, mientras que tampoco existía la idea de usuarios, permisos y privilegios.

8.7.2. Directorio de profundidad fija

El esquema anterior comenzó a presentar problemas a medida que los sistemas se fueron volviendo más complejos. Por un lado con la aparición de sistemas multiusuario se volvió deseable que cada uno pudiese tener su propio directorio, en parte debido a consideraciones de seguridad y en parte para evitar conflictos de nombres[6]. Al mismo tiempo los sistemas operativos mismos comenzaron a tener archivos que debían gestionar, idealmente separados de los archivos de los usuarios.

Esto llevó a la creación de sistemas de archivos con directorios de un solo nivel. Los archivos estaban divididos en directorios, generalmente por tipos y usuario. En este esquema el directorio raíz se llamaba *MFD*, mientras que los directorios derivados se llamaban *UFD*. Bajo este sistema cada usuario solo podía trabajar sobre su *UFD*. Si bien este cambio representó una mejora en general trajo un problema nuevo, era imposible que dos(o más) usuarios compartieran un archivo.

8.7.3. Directorios estructurados en árbol

El siguiente paso natural es permitir directorios de profundidad ilimitada, se permite que cada directorio puede contener archivos y directorios anidados de forma arbitraria. Esto permite que el administrador del sistema y cada usuario estructuren su información de la forma que les resulte lógica y conveniente.

Como una consecuencia natural de este desarrollo surgen las *rutas de búsqueda*, dado que los programas y bibliotecas del sistema pueden estar en cualquier lugar se le debe indicar al sistema donde buscar estos programas y bibliotecas.

8.7.4. Directorios como grafos dirigidos

Si bien puede parecer que los sistemas de archivos modernos utilizan el último esquema mencionado esto no es correcto, utilizan un superconjunto estricto, un grafo dirigido. Bajo este esquema pueden existir múltiples referencias a un archivo(Y por lo tanto directorio). Generalmente los usuarios están restringidos a un grafo acíclico dirigido, para evitar problemas de recursión al buscar en directorios.

La forma en que Unix implementa directorios como grafos dirigidos es mediante los *enlaces*, existen dos tipos:

- **Duros:** Se asocian dos o más archivos compartiendo el mismo `I-nodo`[7]. Es decir que todas las entradas son el mismo archivo y permiten realizar las mismas operaciones, no se pueden crear *enlaces duros* a directorios.

[6] En los sistemas de archivos planos multiusuario los distintos usuarios deben asegurarse de no utilizar el mismo nombre de archivo

[7] Se vera el significado y estructura de un `I-nodo` más adelante

- **Simbólicos:** Es un tipo de archivo especial, meramente apunta a otro archivo, el encargado de resolver este enlace es el sistema operativo. En este caso el archivo y el enlace son dos entidades distintas, si el archivo original es renombrado o eliminado entonces el enlace se rompe.

8.7.5. Archivos especiales de directorio

Tanto en Windows como en Unix hay dos archivos especiales que todo directorio posee, `.` y `..`, aparecen tan pronto el directorio es creado y son esenciales para la navegabilidad el árbol. El `.` es un *enlace duro* al directorio mismo, mientras que `..` es un *enlace simbólico* al directorio padre. Notamos que este comportamiento tiene una excepción en el directorio raíz tanto `.` y `..` apuntan al mismo directorio, si mismo.

8.8. Control de acceso

8.8.1. En Unix

Parte importante de la información que se almacena respecto a cada objeto del sistema de archivos son sus permisos de acceso. Indican al sistema que puede hacerse con cada uno objeto, normalmente diferenciando el acceso dependiendo del usuario. Veremos el modelo tradicional utilizado por Unix.

Desde un principio Unix fue pensado con un esquema multiusuario, bajo esta concepción un usuario pertenece a uno o más *grupos*, estos son gestionados por el administrador del sistema. Cada objeto del sistema de archivos describe sus permisos mediante nueve bits de acceso, junto con identificadores de grupo y usuario. Estos bits están dispuestos en grupos de tres: uno para el *usuario*, uno para el *grupo* y uno para los *otros*. Cada grupo de tres indica si se otorga permiso de *lectura*, *escritura* y *ejecución*; en ese orden. Estos tres grupos permiten suficiente granularidad para expresar casi cualquier combinación de permisos requerida.

Si bien este esquema también aplica a los directorios, que al final del día son archivos, los permisos representan cosas distintas. Los permisos de *lectura* indican que usuarios pueden obtener la lista de archivos contenidos en él, los de *escritura* crear o eliminar archivos y los de *ejecución* quienes pueden entrar al directorio.

8.8.2. Listas de control de acceso

Una desventaja del modelo tradicional de usuarios de Unix es que se requiere la intervención constante del administrador del sistema. Es el encargado de crear y manejar grupos, así como también de encontrar una representación justa para el conjunto de permisos requerido.

Varios sistemas operativos, incluyendo Windows añaden *listas de control de acceso* para intentar remediar este problema. Cada archivo tiene asociada una lista donde se indica que permisos tiene cada grupo y usuario.

Si bien este modelo presenta un control mucho más fino sobre los permisos de cada archivo tiene dos desventajas importantes. En primer lugar la *lista de control de acceso* de cada archivo se almacena separada de su `I-nodo`, pues tiene tamaño variable, esto se traduce en un tiempo de acceso ligeramente mayor. En segundo lugar resulta más difícil presentar de forma concisa y completa los permisos de un archivo dado. Por último este esquema da lugar a ambigüedades, cuya política de resolución debe establecerse previamente; Por ejemplo, si un usuario pertenece a dos grupos, uno con acceso a un archivo y otro sin acceso.

8.9. Fallos y recuperación

A lo largo de la vida de un sistema de archivos, a medida que se crean y eliminan archivos, su tamaño cambia, se montan y desmontan discos, pueden aparecer inconsistencias en su estructura. El tipo de inconsistencias depende de cada sistema y que representación interna utiliza.

Sumado a esto casi todos los discos modernos utilizan controladores inteligentes, estos separan al sistema de la gestión directa de los dispositivos. Esto es relevante pues antes el sistema podía determinar con toda certeza si una operación se había realizado o no, hoy en día los controladores entregan un *acuse de recibo* en el momento que la operación se coloca en el caché del dispositivo; En caso de un fallo de alimentación la operación puede no haber terminado, y aun asi el controlador va a haber enviado el *acuse*.

8.9.1. Datos y metadatos

Los sistemas de archivos modernos se preocupan más en garantizar la integridad de los *metadatos*, los datos que describen la estructura del sistema de archivos; Más que los *datos* en sí, es decir los archivos. El foco está en que una inconsistencia no ponga en riesgo operaciones posteriores o archivos no relacionados. La corrupción y recuperación de datos de archivos dañados, si bien también es importante, recae más bien en el terreno de las aplicaciones de usuario.

8.9.2. Verificación de integridad

Cada sistema operativo incluye programas para realizar la verificación, y si es necesario corrección, de la integridad de su sistema de archivos. Estos programas hacen un barrido del sistema de archivos, buscando inconsistencias. Estas operaciones son procesos intensivos y complejos que pueden demorar horas, dependiendo del tamaño del sistema y la cantidad de archivos. Dado el elevado costo de realizar estas operaciones surgieron esquemas orientados a evitar la necesidad de tener que llamar a este tipo de programas.

8.9.3. Actualizaciones suaves

La idea básica es estructurar el sistema de una forma simple y organizar las escrituras al mismo de forma que el estado resultante no pueda ser inconsistente, ni siquiera en caso de fallo. Al mismo tiempo se exige que todas las operaciones de actualización de datos se realicen de forma síncrona, es decir que solo se informa de su éxito cuando los resultados ya están escritos en el disco.

La implementación de este esquema en la práctica resultó muy difícil, por lo tanto se relajó la exigencia, en lugar de tener un sistema siempre consistente se aceptaron las inconsistencias no destructivas. Desgraciadamente esta estrategia se vio afectada por los controladores inteligentes, el sistema no tenía garantía de que realmente se seguían sus solicitudes, lo que rompe completamente la confiabilidad del proceso. Hoy en día ya casi no se usa.

8.9.4. Bitacora o journaling

Proviene del mundo de las bases de datos, consiste en separar un área del espacio disponible y usarla para llevar una *bitácora*. Esta almacena todas las *transacciones*, es decir operaciones que aparecen atómicas, que se deben realizar sobre metadatos. Periódicamente el sistema verifica las operaciones pendientes y *avanza* sobre la bitácora, realizando las operaciones y marcando cada una a medida que las realiza.

En caso de tener que recuperarse de una condición de fallo el sistema solo debe leer la bitácora y aplicar las *transacciones* no realizadas. Una restricción sobre este esquema es que las *transacciones* guardadas en la bitácora deben ser *idempotentes*, es decir, el efecto de realizarla dos veces debe ser el mismo que si solo se realiza una vez.

Generalmente se implementa utilizando una lista ligada circular, con un puntero que indica cuál fue la última operación realizada exitosamente. Es el esquema más utilizado hoy en día.

8.9.5. Sistemas estructurados en bitácora

Si se designa todo el espacio de almacenamiento como la bitácora el resultado es un sistema de archivos *estructurado en bitácora*. Este esquema presenta una organización completamente distinta a los sistemas de archivos tradicionales.

En estos sistemas todas las modificaciones, tanto de *datos* como *metadatos*, se escriben de forma secuencial a la *bitácora*. Para esto suelen utilizar de forma agresiva un cache de gran capacidad, acompañado de un fuerte mecanismo de recolección de basura, acomodando la información y liberando todo lo que resulte redundante.

9. Sistemas de archivos

9.1. Discos duros

Como se explicó en la sección anterior el objetivo principal de un sistema de archivos es almacenar información de forma persistente. La forma de hacer esto en un equipo moderno es mediante dispositivos que almacenan bloques, es decir conjuntos de N bytes, generalmente de $1KB$ o $0{,}5KB$; La persistencia se ocurre a nivel de estos sectores.

Nos vamos a concentrar en los discos duros como dispositivo de almacenamiento de bloques, existen otros medios persistentes, pero escapan el alcance de esta materia. Físicamente un disco se particiona siguiendo una geometría de bloques, compuesta por varias estructuras:

- **Platos:** Son el componente básico de un disco duro, todas las estructuras nombradas a continuación residen en él, se pueden registrar datos en ambos lados del mismo. Los discos modernos tienen una gran cantidad de platos, apilados verticalmente.
- **Pistas:** Cada superficie de un plato está dividida por una serie de círculos concéntricos, su centro coincide con el del plato. La pista cero está en el borde exterior.
- **Sector:** Es la unidad básica de almacenamiento de en un disco duro, son el bloque. Es un segmento de arco de una de las pistas, el número de sectores es variable.
- **Cilindro:** Es un conjunto de varias pistas, ubicadas en distintos platos y alineadas verticalmente, estando una en cada cara de cada plato.

Para leer la información contenida en los sectores un disco posee cabezales de lectura, uno por cada lado de cada plato, estos se mueven en conjunto. Dentro del disco la velocidad de rotación es mayor a la de los cabezales de lectura, por lo tanto es más rápido leer sectores contiguos que sectores en cilindros distintos.

En un principio el sistema operativo accedía al disco indicando la ubicación de un bloque de información particular, utilizando la notación *CHS*(Cylinder, Head, Sector). A principios de los noventa se transicionar al esquema de *direccionamiento lógico de bloques*(*LBA*). Este cambio permitió ganar rendimiento y aprovechar mejor el espacio en el disco, a costa de que el sistema operativo perdiese el control directo del disco[8].

9.2. Estructura

9.2.1. Componentes

Los sistemas de archivos utilizan los siguientes conceptos para almacenar, ubicar y recuperar información:

- **Partición:** Una subdivisión del disco, que le permite al administrador del sistema o usuario definir cómo se emplea ese espacio en el disco. Un disco puede tener múltiples *particiones*, cada una con un sistema de archivos independiente.

[8] Este cambio vino de la mano con los controladores de disco inteligentes mencionados en la sección anterior.

- **Volumen:** Colección de bloques que poseen un sistema de archivos, presentados al usuario como una unidad normalmente un *volumen* coincide con una partición[9].
- **Metadatos:** Formalmente es toda información acerca de un archivo que no es el contenido del archivo en sí. Por ejemplo su nombre, tamaño, propietario, etc.

9.2.2. El directorio raíz

En la sección anterior se mencionó la existencia del directorio raíz, formalmente el directorio raíz es el *comienzo* del sistema de archivos, es el directorio de más alto nivel. Todos los demás archivos y directorios se estructuran a partir de él, está siempre en una ubicación bien conocida dentro del sistema de archivos.

Al mismo tiempo relaciona los archivos como son presentados al usuario, con una ruta y un nombre con las estructuras que los describen ante el sistema de archivos, los *I-nodos*.

9.3. Algunos sistemas de archivos

9.3.1. Unix FS

Se comenzó a desarrollar a principios de los setenta, es la base de la mayor parte de los sistemas de archivos basados en los derivados de Unix, su concepto principal es el *I-nodo*.

Dado un volumen con *Unix FS* la distribución del espacio, en orden es:

1. **Bloque de arranque:** Espacio reservado para el booteo del sistema, contiene el firmware `UEFI`/`BIOS`.
2. **Superblock:** Contiene información sobre todos los parámetros del disco, como está formateado, información de *I-nodos* libres, última fecha de montaje, etc. Un miembro crucial es el **mapa de bits de espacio libre**, indica cuales son los bloques libres del disco, en esta estructura cada bloque se representa con un bit; Permite encontrar la cantidad de espacio ocupado y disponible de forma compacta.
3. **Tabla de *I-nodos*:** Posee información sobre todos los *I-nodos* ocupados del sistema; Son la estructura en disco que guarda los *metadatos* de cada archivo, excepto el nombre, que está en el directorio.
4. **Directorio raíz:** Contiene todas las *dirEntries* del directorio raíz, en este sistema de archivos son muy sencillas, debido a que la mayor parte de la información se almacena en los *I-nodos*.

Un poco más sobre *I-nodos*: Bajo este sistema de archivos los *I-nodos* generalmente poseen un tamaño fijo, 512 bytes, como ya se explicó poseen toda la metadata de un archivo en distintos campos. Esto incluye el tamaño del archivo, sus permisos, dueños, *links*, etc.

Un campo de particular interés es el de *links*, como esta estructura no almacena el nombre del archivo, un archivo puede tener distintos nombres(Ver sección 8.7.4), lo que importa es que cada uno haga referencia al *I-nodo* correspondiente en el disco. Este campo indica cuantas referencias tiene el archivo, si llega a 0 significa que el archivo puede borrarse, pues no quedan más referencias a él. Este campo trata sobre los *links duros*.

Finalmente, el *I-nodo* contiene una serie de listas de sectores del disco donde está el archivo. Algunas de estas listas apuntan a sectores de forma directa, mientras que otras apuntan a listas de sectores o listas de listas de sectores. Esto permite almacenar archivos de gran tamaño de forma eficiente.

Un poco mas sobre *dirEntries*: Almacenan el nombre de un archivo y su *I-nodo*, esto evita la información duplicada, y significa que la información sobre el archivo no depende del directorio donde se encuentra. Al *borrar* un archivo realmente se está eliminando su *dirEntry*.

[9] Existen ciertas excepciones, escapan el alcance de la materia.

Como crear un archivo: VER

Como borrar un archivo: Borrar un archivo en *Unix FS* es sencillo:

1. Se busca el archivo en el directorio correspondiente para obtener su *I-nodo*.
2. Se elimina la *dirEntry* del directorio del directorio correspondiente.
3. Se verifica la cuenta de *links* del *I-nodo*, si es 0 el archivo se borra. Para esto el *I-nodo* se marca como libre en la tabla de *I-nodos* y se marcan como libres en el bitmap todos los sectores en el *I-nodo*.

En caso de que ocurra una falla entre el paso 2 y 3 se obtiene un *I-nodo huérfano*, que no es referenciado desde ningún directorio. Esto no es tan grave, existen mecanismo que permiten verificar el disco para encontrar *I-nodos* huérfanos. Cuando esto ocurre se puede intentar recuperar el archivo en cuestión, alternativamente el sistema puede decidir reclamar el espacio asumiendo que pertenece a un archivo borrado.

9.3.2. TAR(Unix)

Se creó para trabajar con almacenamiento en cintas magnéticas, los datos y metadatos se almacenan de forma secuencial, en bloques de 512 bytes. Todo archivo es precedido por una cabecera, de un bloque, que contiene la metadata. En caso de que los datos de un archivo no lleguen a ocupar bloques completos se utiliza relleno. Los bloques no se comparten entre distintos archivos. Notas:

- Muy simple de implementar.
- Ineficiente a la hora de leer y modificar archivos.
- Buscar un archivo implica recorrer toda la lista de archivos.

9.3.3. FAT

Formalmente se llama *MSDOS FS*. Los datos no se organizan en sectores, sin en *clusters*, formados por grupos de sectores, cada *cluster* contiene 4 sectores y cada sector 512 bytes; Por lo tanto cada *cluster* tiene 2048 bytes o 2 KB. La idea de este sistema era unificar el manejo del espacio libre y la fragmentación. Vale la pena notar que bajo este esquema las escrituras deben ser *sincronas*(Ver sección 8.9.3) para evitar que ocurran problemas de inconsistencia.

Bajo este esquema un *cluster* es la unidad mínima lógica de almacenamiento. Por ejemplo si sólo se desean almacenar 10 bytes, se les asignará un *cluster*, 2048 bytes. Esto implica que *FAT* es susceptible a fragmentación interna.

Para implementar esto se utiliza una estructura denominada *File Allocation Table(FAT)*, de donde proviene el nombre más común del sistema. En esta cada *cluster* es representado por enteros de N bytes, donde N depende del tamaño de la *FAT*[10]. Los enteros pueden tener los siguientes valores:

- **0:** El *cluster* esta libre.
- **F ... FE:** *Cluster* dañado, seteado durante el formateo.
- **F ... FF:** Ultimo *cluster* de un archivo.
- **Otro:** *Cluster* en uso por un archivo, el número indica el próximo *cluster* del archivo. De esta forma los archivos se almacenan como una lista enlazada simple de *clusters*.

[10] De aquí provienen los nombres de las distintas versiones *FAT-12*, *FAT-16*, *FAT-32*.

Vale la pena notar que *FAT* no analiza que *clusters* utilizar cuando los elige para almacenar un archivo, lo que lleva a una de sus grandes debilidades, su tendencia a fragmentarse. Esto causa que un archivo pueda estar esparcido a lo largo de todo el disco, haciendo lento y costoso operar sobre él. Al mismo tiempo la *FAT* ocupa mucho espacio y hay que mantenerla siempre en la memoria principal, algo particularmente problemático a medida que fue aumentado el tamaño de los discos.

Dado un volumen con *FAT* la distribución del espacio, en orden es:

1. **Bloque de arranque:** Espacio reservado para el booteo del sistema, contiene el firmware `UEFI`/`BIOS`.
2. **1° FAT:** Tabla *FAT* primaria.
3. **2° FAT:** Tabla *FAT* secundaria, todo cambio realizado a la *FAT* primaria se ve espejado en esta. Es utilizada en caso de que la primera tabla sea inconsistente o tenga datos corruptos.
4. **Directorio raíz:** Contiene todas las entradas del directorio raíz, en este sistema de archivos son complejas, muy similares a un *I-nodo*.

Un poco más sobre entradas del directorio: Este sistema de archivos no separa entre entre entradas del directorio e *I-nodo*, son la misma cosa. Contiene entonces una combinación de la información que uno esperaría encontrar en cada estructura: nombre del archivo, extensión, permisos, tamaño del archivo, primer cluster, etc. Formalmente un subdirectorio es un archivo especial, al mismo tiempo el directorio raíz tiene un tamaño limitado, pues está ubicado en una sección fija del disco.

Dado que los directorios son archivos, son susceptibles a fragmentación. Esto puede causar grandes problemas de rendimiento al recorrer directorios, especialmente si tienen múltiples niveles de profundidad.

Como crear un archivo: La creación de archivos en *FAT* no es atómica, requiere:

1. Encontrar una lista de *clusters* suficiente para almacenar la información del archivo.
2. Encontrar el siguiente espacio disponible en el directorio.
3. Crear la entrada del directorio en el espacio determinado en el paso 2.
4. Almacenar los datos del archivo en los *clusters* determinados en el paso 1.

Cualquier fallo después del paso 3, la primera modificación, hará que el archivo se corrompa y que el sistema de archivos termine en un estado inconsistente.

Como borrar un archivo: Cuando se le solicita a un sistema de archivos *FAT* que borre un archivo este hace algo diferente a lo que uno esperaría. No se elimina la entrada del directorio ni tampoco las entradas en la *FAT*. En lugar de eso se marca la entrada en el directorio y los *clusters* correspondientes como disponibles. Esto se hace debido a que borrar todas las entradas correspondientes a un archivo es lento y costoso.

Control de acceso: Los sistemas *FAT* realizan control de acceso mediante un byte en la entrada del directorio de cada archivo, los valores posibles son:

- **Oculto:** El sistema no debe mostrarle el archivo al usuario.
- **Solo lectura:** El sistema debe evitar la modificación del archivo.
- **Sistema:** Archivos especiales del sistema operativo, no deben moverse ni fragmentarse.
- **Archivado:** Un mecanismo primitivo para realizar *backups*, cada vez que se realizaba un respaldo el archivo era marcado como *archivado*, si era modificado se retiraba el atributo. Esto le permite al sistema solamente archivar los archivos modificados desde el último respaldo.

9.4. Mejoras

El sistema de archivos introducido por *BSD*, basado en *Unix FS* introdujo muchas mejoras, principalmente al rendimiento, algunas de ellas son:

9.4.1. Haciendo crecer el sector lógico

BSD introdujo el concepto de sectores *lógicos*, distintos de los sectores *físicos* del disco. Un sector *lógico* tenía un tamaño de 4096 bytes, contra los 512 bytes de un sector *físico*.

Esto aceleró significativamente los accesos al disco, pues cuando un programa pedía un sector *físico* se traían todos los del sector *lógico*, de esta forma cuando el programa perdía el próximo sector ya se encontraba en memoria. Aportó una mejora en velocidad del 20 %.

9.4.2. Algoritmo del ascensor

Vale la pena notar que este término se refiere a una familia de algoritmos: *SCAN*, *LOOK* y *C-SCAN*, solo analizaremos el primero. La idea de este algoritmo es minimizar la cantidad de movimiento que debe realizar el cabezal para realizar una operación, intenta minimizar los cambios de pista realizados. Para esto se escriben todos los accesos necesarios en un caché, donde se ordenan antes de realizar el acceso al disco, evitando largos recorridos; Esto también le da su nombre, debido a que recorre las pistas de la misma forma que lo hace un ascensor.

Vemos un ejemplo práctico, notar que deberiamos usar notacion *CHS*, dado que solo nos interesa el rendimiento podemos ignorarlo. Sea la secuencia de acceso:

$$\langle 1, 10, 5, 2, 9, 3 \rangle$$

Calculando las distancias a recorrer sin utilizar el algoritmo, obtenemos:

$$1 \to 10, 10 \to 5, 5 \to 2, 2 \to 9, 9 \to 3 \equiv 9 + 5 + 3 + 7 + 6 = 30$$

Reordenando ahora los accesos utilizando el algoritmo:

$$\langle 1, 2, 3, 5, 9, 10 \rangle$$

Calculando las distancias a recorrer utilizando el algoritmo, obtenemos:

$$1 \to 2, 2 \to 3, 3 \to 5, 5 \to 9, 9 \to 10 \equiv 1 + 1 + 2 + 4 + 1 = 9$$

Vemos así que el algoritmo del ascensor presenta una mejoría significativa, sin embargo no es óptimo. Solo es óptimo si los pedidos de lectura y escritura son independientes.

Ahora bien esta familia de algoritmos, y los algoritmos de planificación de acceso en general casi no se utilizan hoy en día, esto se debe a tres motivos principales:

- Solo consideran el movimiento del cabezal, ignorando la demora rotacional, que representa una porción considerable de la demora al realizar operaciones.
- Los sistemas operativos modernos desean darle distintas prioridades a solicitudes de acceso al disco, algo que estos algoritmos no pueden hacer
- El punto más importante, los controladores inteligentes de disco. Dado que la interacción del sistema con los discos es cada vez más abstracta el sistema carece de la información necesaria para planear correctamente sus accesos.

9.4.3. Distribucion del superblock

A medida que se usa el disco, estando en cualquier parte del mismo, va a ser necesaria información del *superblock* y la tabla de *I-nodos*, que se encuentran al principio del disco. Esto generaba una gran cantidad de tiempo perdido, yendo desde y hacia los datos.

Para solucionar esto *BSD* optó por distribuir estas estructuras a lo largo del disco, con una noción de localidad. Es decir, se tiene el *superblock*, bitmap y tabla de *I-nodos* específicos a un grupo de cilindros cerca de estos. Esta distribución no se hace a múltiplos exactos, de forma que si se rompe alguna pista o cabezal no se pierdan todos los datos.

El sistema de archivos implementando esta mejora se llamó *FFS*, aportó una mejora en velocidad del 60 %.

9.4.4. Eliminando secuencias dependientes

Como se nota que el algoritmo del ascensor se rompe en caso de que las operaciones sean dependientes entre sí, se pueden intentar eliminar dichas dependencias para usar el algoritmo de forma plena. Esto fue introducido por los sistemas de archivos *ext* y *ext2*.

9.4.5. Cambiando la estructura del directorio

Acceder a la lista de *dirEntries* de un directorio es lento, puesto que es un array $\rightarrow O(n)$. Una primera mejora fue utilizar árboles balanceados $\rightarrow O(log(n))$. Esta cota se mejoro aun mas pasando a tablas hash $\rightarrow O(1)$.

Si bien esta mejora incrementa significativamente la eficiencia también añade algunas complicaciones respecto a la modificación del directorio. Con una lista toda modificación tiene costo $O(1)$, esto es más costoso en una tabla hash y requiere ciertas consideraciones adicionales, sin embargo en promedio utilizar tablas hash es más eficiente.

9.5. Algunos sistemas de archivos actuales

9.5.1. ext4

Es una mejora sobre *ext3*, ya está quedando obsoleto, pues es un sistema de parches sobre parches(*ext3* es una mejora sobre *ext2* y *ext2* una mejora sobre *extFS*). Notas:

- Soporta volúmenes de hasta 1024 PiB.
- Soporta extents, una forma de alocar espacio contiguo para almacenamiento.
- Menor uso del CPU.
- Mejoras en la velocidad de lectura y escritura.
- Utiliza journaling, como su predecesor(*ext3*).
- Soporta *snapshots*, son "fotos" del estado del sistema.

9.5.2. B-tree FS

Se posiciona como el sucesor de *ext4*, como lo indica su nombre se basa en árboles binarios. Notas:

- Soporta *snapshots*.
- Soporta *copy-on-write*.
- Tiene optimizaciones para discos SSD.
- Es un sistema de archivos completamente nuevo, sin los parches de *ext*.